

Smart Car Washing Center using IoT

By:

Student Name: **Hisham Abdulfattah Idrees, ID:202411240**

Project

Submitted to

King Fahd University of Petroleum & Minerals

College of Computer Sciences and Engineering

Department of Computer Engineering

In partial fulfillment of the requirements for the degree of

Master of Science in

Internet of Things and Embedded Systems

Supervised By:

Dr. Khaled Rabia

April/May 2026

Table of Contents

Abstract	7
Acknowledgment	8
1 Introduction.....	9
2 Literature Review.....	10
2.1 Introduction to IoT and Automation in Car Wash Systems.....	10
2.2 Core IoT Technologies in Car Wash Automation	11
2.2.1 Sensor Technologies	11
2.2.2 Controller Architecture and FreeRTOS	11
2.2.3 Communication Protocols.....	11
2.3 Recent Technological Advancements	12
2.3.1 Real-Time Monitoring and Dashboards.....	12
2.3.2 RFID-Based Authentication.....	12
2.4 Industry Practices and Commercial Context.....	12
2.5 Gaps and Research Opportunities	12
2.5.1 Technical Limitations of Existing Systems	12
2.5.2 Functional and Scalability Gaps	13
2.5.3 Summary Comparison Table	13
3 Problem Statement.....	14
3.1 Aim and Objectives.....	15
3.2 Scope and Limitations.....	15
4 System Design	16
4.1 System Architecture.....	16
4.2 Hardware Design	17
4.2.1 Microcontroller ESP32-S3.....	17
4.2.2 RFID Reader MFRC522.....	18
4.2.3 Water Quality Sensors	18
4.2.4 Actuators and Relay Control.....	18
4.2.5 Power Supply	19
4.2.6 Environmental Sensor	19
4.3 Software Design.....	21
4.3.1 Embedded Firmware ESP32-S3	21
4.3.2 Cloud Backend Flask + Google Cloud	21

4.3.3	Frontend React Web Application.....	22
4.3.4	Dashboard Home	22
4.3.5	Dashboard Last Report	23
4.3.6	Dashboard Revenue and Consumption.....	24
4.3.7	Dashboard System Data.....	25
4.3.8	Users Home.....	25
4.3.9	Users Add Users	26
4.3.10	Users Block Users.....	26
4.3.11	Users Add Balance.....	27
4.3.12	Analytics	28
4.4	Communication Protocols.....	28
4.5	System Operation Flow.....	29
4.5.1	Health equipment Check.....	29
4.5.2	Mission Flow	30
4.6	Mathematical Models and Formulas.....	32
4.6.1	Firmware Mathematical Models	32
4.6.2	Backend Business Logic Models	34
5	Performance Evaluation:.....	36
5.1	Performance measures	36
5.2	Experimental/simulation design:	36
5.3	Results and Discussion:	37
5.3.1	Water Consumption Analysis	37
5.3.2	RFID Authentication and Wallet Protocol.....	38
5.3.3	Sensor Accuracy	39
5.3.4	Loyalty Program and System Stability	39
6	Conclusion and Future Directions	40
6.1	Conclusion	40
6.2	Future Directions	41
6.2.1	Closed-Loop Water Recycling.....	41
6.2.2	Reverse Osmosis (RO) Treatment	42
6.2.3	Greywater Integration	42
6.2.4	AI-Based Predictive Analytics.....	42
6.2.5	Multi-Station Cloud Federation	42
6.2.6	Automatic Plate Recognition (ALPR)	42

6.2.7	Edge AI	42
6.2.8	Combined Impact Target	42
6.3	Challenges Encountered.....	43
6.3.1	Electromagnetic Interference (EMI) from DC Pump Motors.....	43
6.3.2	Wi-Fi Signal Degradation Inside Metal Enclosure	43
6.3.3	MQTT-TLS Certificate Memory Management	44
6.3.4	Google Cloud Run Cold-Start Latency.....	44
6.3.5	TDS Sensor Temperature Drift.....	45
6.3.6	ADC Noise on TDS Analog Line	45
6.4	Lessons Learned.....	46
7	References	47
8	Appendices.....	50
8.1	Source Code	50
8.2	Appendix A: ESP32-S3 Pin Assignment.....	50
8.3	Appendix B: MQTT Topic Structure.....	51
8.4	Appendix C: Service Pricing and Loyalty	51
8.5	Appendix D: Firmware Key Code Sections.....	51
8.6	Appendix E: Backend REST API Reference.....	52

List of Figures

Figure 1: BAREEQ System Architecture Hardware, Cloud, and Frontend Layers.....	16
Figure 2: BAREEQ System Architecture	17
Figure 3: ESP32-S3 Microcontroller	17
Figure 4: Full Prototype Assembly	17
Figure 5: MFRC522 RFID Reader Module	18
Figure 6: TDS Sensor Module	18
Figure 7: YF-S201 Water Flow Sensor	18
Figure 8: MAX6675 Thermocouple Amp	18
Figure 9: 12V DC Diaphragm Pump	19
Figure 10: 4-Channel Relay Module (Wired)	19
Figure 11: 12V SMPS Power Supply	19
Figure 12: DC-DC Buck Converter (HW-688)	19
Figure 13: AHT20 Temperature and Humidity Sensor	20
Figure 14: Dashboard Home	23
Figure 15: Dashboard Last Report.....	23
Figure 16: Dashboard Revenue and Consumption	24
Figure 17: Dashboard Monthly Revenue and Consumption Table	24
Figure 18: Dashboard System Data	25
Figure 19: Users Home	26
Figure 20: Users Add Users.....	26
Figure 21: Users Block Users	27
Figure 22: Users Add Balance	27
Figure 23: Analytics Water Quality, Water Flow	28
Figure 24: BAREEQ System Operation Flowchart	31

List of Tables

Table 1: Comparison of BAREEQ with Related IoT Car Wash Systems	13
Table 2: BAREEQ Hardware Component Summary	20
Table 3: BAREEQ Cloud Database Schema	22
Table 4: BAREEQ Communication Protocol Summary	29
Table 5: Water Consumption Comparison - Manual vs. BAREEQ	37
Table 6: Water consumption Manual vs BAREEQ	38
Table 7: BAREEQ Communication Latency Measurements	38
Table 8: BAREEQ Sensor Accuracy Results	39
Table 9 Summary of Engineering Challenges and Resolutions.....	46
Table 10: ESP32-S3 Pin Assignment for BAREEQ Prototype	50
Table 11: BAREEQ MQTT Topic and Payload Structure	51
Table 12: BAREEQ Service Codes and Pricing	51
Table 13: BAREEQ Backend REST API and WebSocket Endpoint Reference	52

Abstract

IoT has grown fast in recent years, and this opens up new ways to automate service businesses so they run smoother, with more transparency, and with less waste. The Smart Car Washing Center project (BAREEQ) is one example. It is a fully automated, IoT-based car wash for indoor stations. The system brings real-time monitoring, cloud computing, contactless authentication, and automatic actuator control together into one platform.

The hardware uses an ESP32-S3 microcontroller running FreeRTOS firmware with three tasks: one for RFID reading, one for relay control, and one for sensor sampling. An MFRC522 reader handles contactless user verification. A set of sensors TDS, a YF-S201 flow meter, an AHT20 temperature/humidity sensor, a MAX6675 thermocouple, and a turbidity sensor keeps watch on the operating and water-quality numbers during each wash. Two DC diaphragm pumps act as actuators, driven through two 4-channel relay modules. Every wash mission follows the same fixed sequence: 30 s of water pumping, a 5 s break, then 30 s of air drying. Total time is 65 s.

For communication, the device talks to a HiveMQ Cloud broker over MQTT with TLS on port 8883. This keeps the messages secure and lightweight. The backend is a Python Flask app on Google Cloud Run. It subscribes to the MQTT topics, processes wallet payments atomically, and saves all sensor data and mission summaries into a Google Cloud SQL (MySQL) database. It also exposes REST and WebSocket endpoints so the frontend can pull and stream data.

On the frontend side, a React web app gives the manager a live dashboard that refreshes every second through WebSocket. It also offers a user management page for RFID registration and balance handling, plus an analytics page showing monthly and yearly revenue, water consumption, TDS trends, and the service mix. The business logic layer adds two extras: a loyalty program that gives one free wash after every seven paid ones, and configurable per-service discounts.

Compared with manual car washing, BAREEQ shows clear improvements. By using high-pressure atomized spray nozzles together with sensor-driven dosing, the prototype cuts fresh water use per vehicle by about 70 to 80 percent from the typical 100 to 300 L of manual washing down to 30 to 50 L. In the lab tests, the average MQTT round-trip time was 276 ms (between 240 and 372 ms across 20 trials). Every sensor stayed inside its rated tolerance over a 24-day field test, and the FreeRTOS watchdog never reset once during a 6-hour stability run that covered 15 wash cycles. The project lines up with Saudi Vision 2030 and the National Water Strategy 2030, and the modular cloud design leaves room to grow toward a closed-loop recycling target of under 15 L per car.

Acknowledgment

All praise and gratitude belong to Allah, whose boundless blessings made this achievement possible granting me the opportunity to defend this research project for the degree of Master of Science in Internet of Things and Embedded Systems, Department of Computer Engineering, at **King Fahd University of Petroleum and Minerals (KFUPM)**. I owe a profound debt of gratitude to KFUPM, an institution that has shaped engineers and innovators for generations for its rigorous academic environment, world-class faculty, and steadfast commitment to applied research excellence, all of which provided the intellectual foundation upon which this work was built. I extend my deepest and most sincere gratitude to **Dr. Khaled Rabih**, Project Supervisor, for his unwavering scientific guidance, continuous encouragement, and dedicated support throughout every phase of this project, and to **Dr. Ashraf Mahmoud** for his invaluable time, thoughtful evaluation, and the depth he brought to the defense discussion. **To every instructor at KFUPM** who challenged me thinking and invested time beyond the classroom this work carries your influence. I dedicate the fruits of this effort to the soul of my late father, **Abdulfattah Mahmoud Idrees** may Allah have mercy upon him who was the finest example I ever knew, whose prayers and wisdom continue to illuminate my path, and to my beloved mother **Ihsan Muhammad Taha**, whose prayers have always been my compass and inner strength. To my dear wife **Ashwag** my partner in every step and to my children **Abdulfattah**, **Lma**, and my little one **Liyan**: I humbly ask your forgiveness for the time I could not fully give during these years of study your patience and love sustained me throughout. My heartfelt thanks also go to my brothers **Muhammad**, **Hitham**, and **Hiyam** for their sincere prayers and moral support, and to my colleague **Eng. Hadi Al-Mansour** for his dedicated technical support, to **Eng. Ahmed Al-Husseini** for enriching this project with his field expertise, and to **Eng. Fahd Al-Hamzi** whose encouragement was a constant driving force from the very beginning. All praise is due to Allah, by whose grace all good deeds are completed.

1 Introduction

Modern car wash centers are now expected to run with more automation, better accountability, and higher efficiency. Older systems use manual steps, fixed timers, and local-only data, so they can't really track resource use, prove that a service was delivered, or offer customers any digital experience. IoT changes that. Connected sensors, cloud computing, and automated control have already reshaped many service industries [1, 5]. The car wash sector is now heading the same way. Today's customers want fast service, easy payment, and a clear breakdown of what they're paying for. Owners, on their side, want a way to track water use, cut waste, and manage more than one branch from a single screen. That mix of needs is what makes IoT-based solutions not just nice-to-have but essential for the future of this business.

Water scarcity is a serious issue in Saudi Arabia and across the wider Middle East. Car wash facilities are among the bigger water consumers in the commercial service sector. Industry numbers show that in-bay car washes without reclamation use roughly 55 to 75 gallons of fresh water per vehicle, and that smart sensor-driven control can bring this down by a meaningful margin. This is the main motivation for building a system that measures water flow, water quality, and overall use in real time [17]. Studies on IoT-based smart water management in Saudi Arabia confirm the same point: sensors plus cloud connectivity are what will let the Kingdom hit its Vision 2030 water targets [22]. Saudi Arabia is also one of the most water-stressed countries on the planet. Most of its drinking water comes from desalination, which is expensive and energy-hungry. So even small per-wash savings, multiplied across hundreds of stations nationwide, add up to something significant. A smart system that logs every liter used can prove, in numbers, how much water is being saved each day, each month, and each year.

Water volume is only part of the story. Manual washing, especially of diesel vehicles and heavy fleets, produces wastewater loaded with hydrocarbons, engine oil, grease, brake-dust metals, and detergent. When that water just drains into storm drains or onto the ground, it puts groundwater and aquatic life at real risk. To put it in perspective, a single liter of waste oil can contaminate up to one million liters of drinking water. In a water-poor region like Saudi Arabia, that makes uncontrolled wash runoff a serious environmental hazard [16, 17]. A smart wash setup can help here by recording how much water is used and how dirty it gets during each cycle. That data is the starting point for proper treatment and recycling later on. It turns a harmful waste stream into something controlled and trackable, and it helps owners stay on the right side of local environmental rules.

IoT lets physical systems collect, send, and act on data in real time, using connected devices and cloud services. Applied to car washing, that means: vehicles get verified without manual intervention, the operating numbers are watched remotely, payments happen automatically, and

analytics from many stations land in one place. Together these features close the main gaps of older systems wasted resources, lack of transparency, weak customer experience, and missing data for optimization [15, 5]. With cheap microcontrollers, secure cloud services, and easy wireless protocols all widely available now, building a smart car wash is more practical and more affordable than ever before. Small and mid-sized businesses can adopt it too, not only the large chains. An owner can start with one station and add more later, step by step, without rebuilding the system.

This project, called BAREEQ (Smart Car Washing Center using IoT), covers the design and implementation of a fully automated IoT-based car wash. The system runs on an ESP32-S3 with FreeRTOS, communicates with a cloud backend using MQTT over TLS, stores everything in Google Cloud SQL, and shows the operational view through a React-based web dashboard. The user is identified by an MFRC522 RFID reader. The wash itself follows a fixed sensor-monitored sequence: water washing, a short break, then air drying. The whole thing is meant to be simple to use, easy to expand, and ready to scale into a network of stations across multiple cities. It also gives owners and managers a clear picture of how each station is performing, so they can make better decisions and improve service over time. The use of secure cloud links and live sensor data makes the platform reliable enough for long-term operation.

The rest of this report is structured as follows. Section 2 reviews the relevant literature on IoT-based car wash systems, water monitoring, RFID authentication, and embedded systems. Section 3 covers the problem statement and project objectives. Section 4 walk through the full system design hardware, software, and communication. Section 5 evaluates performance using a defined set of metrics and experimental results. Section 6 closes the report with conclusions and future directions. Literature Review

2 Literature Review

2.1 Introduction to IoT and Automation in Car Wash Systems

IoT has slowly but steadily turned car wash systems from simple timer-based relay boxes into smart, connected platforms that can monitor in real time and make data-driven decisions. The earliest work in this space focused mainly on the hardware side using microcontrollers and relay modules to sequence wash steps, but without any cloud link or sensor feedback [5].

One of the early IoT-based car wash centers was proposed by Kshirsagar et al. They used an ATmega328 with an ESP8266 Wi-Fi module to automate a conveyor-style wash and sent basic logs to a MySQL database. The system worked, but it had no water-quality sensing, no secure

communication, and no payment or loyalty handling at all [11]. quality monitoring, secure communication, and any form of user payment or loyalty management [11].

More recently, Hossain et al. built an automatic wash using a NodeMCU and the Blynk IoT platform. Infrared proximity sensors triggered the wash stages, and a mobile app gave the user interface. The downsides: no water-quality sensing, no real payment system, and a hard dependency on the Blynk cloud, which limits custom analytics [12].

2.2 Core IoT Technologies in Car Wash Automation

2.2.1 Sensor Technologies

Hall-effect flow sensors like the YF-S201 are a common pick in IoT water monitoring. They count pulses to measure volume, which is a simple and cheap way to track water use in real time [18].

TDS-based water quality sensing has also been used in many IoT projects. Studies report that TDS sensors covering the 0 to 1000 ppm range, paired with cloud logging, can hit better than 96 percent accuracy in controlled conditions [19]. A broader survey of IoT water management work also confirms that TDS and turbidity, when combined with cloud analytics, are the two most common parameters used for real-time water quality checks in smart infrastructure [24].

2.2.2 Controller Architecture and FreeRTOS

ESP32 microcontrollers have become the go-to platform for IoT embedded work. They come with a dual-core CPU, built-in Wi-Fi and Bluetooth, and a healthy set of peripherals SPI, I2C, ADC, and interrupt-capable GPIO. FreeRTOS sits on top, providing preemptive task scheduling and tight timing control. Several studies show that priority-based scheduling cuts sensor sampling jitter a lot compared to single-threaded polling [13, 14, 20].

2.2.3 Communication Protocols

MQTT works on a publish-subscribe model with a central broker. That fits IoT use cases well, since communication between devices and the cloud is usually one-way and bursty. Running MQTT over TLS gives both the lightweight messaging part and strong security through certificate-based encryption [6, 20]. Smart-city deployments using MQTT end-to-end have shown reliable, low-latency delivery between constrained edge devices and cloud backends, which is why MQTT is now the protocol of choice for real-time service automation [25]. Work on MQTT QoS modeling also confirms that the right QoS settings cut both message loss and latency variation in city-scale IoT setups [26].

2.3 Recent Technological Advancements

2.3.1 Real-Time Monitoring and Dashboards

Real-time dashboards are central to making IoT-based services transparent. Studies on MQTT-driven WebSocket dashboards report sub-second update latency, which lets operators react to anomalies the moment they happen [17].

Water-quality systems that pair IoT sensors with cloud storage have shown clear value: continuous logging gives operators a real audit trail. Setups that combine TDS, pH, dissolved oxygen, and temperature with a cloud platform deliver real-time alerts and historical trends, which catch quality drops far faster than periodic manual sampling does [15, 19]. IoT-based wastewater systems for smart cities go even further. They use sensor-driven classification together with cloud connectivity to separate treated water for agricultural reuse, lowering contamination risk in the urban water cycle [21, 23].

2.3.2 RFID-Based Authentication

RFID is used widely in IoT for contactless authentication and payments. The MFRC522 module, working at 13.56 MHz, shows up in countless embedded systems for user verification and access control. Studies report that RFID-based payment setups exceed 99 percent transaction success at read distances of 1 to 3 cm.

2.4 Industry Practices and Commercial Context

On the commercial side, car wash operators that use smart sensor control and water reclamation report water savings of 50 percent or more over standard timer-based setups. The International Carwash Association's Water Savers program suggests keeping fresh water use under 40 gallons per vehicle as a baseline target [17].

The business side is changing too. Car wash operators are moving toward subscription plans and loyalty-based service. Digital tools that track how often a customer comes back and how much they spend make it possible to run targeted promotions and balance station capacity more carefully [1, 4].

2.5 Gaps and Research Opportunities

2.5.1 Technical Limitations of Existing Systems

Looking at the existing IoT car wash prototypes side by side, a few common gaps stand out. None of the reviewed systems do real-time water-quality monitoring during the wash. TDS and turbidity are simply not measured. Most also rely on plain HTTP or basic MQTT

without TLS, which leaves real security holes around payment data and user information [3, 11, 12].

2.5.2 Functional and Scalability Gaps

The other gap is on the payment side. None of the existing works build a real-time MQTT wallet protocol with atomic database transactions, duplicate-request protection, and loyalty rewards baked in. Multi-station scaling is also missing every prototype reviewed is a single-station setup with no path toward a unified cloud platform [9, 5].

2.5.3 Summary Comparison Table

Table 1 summarizes the key differences between this project and the most closely related works in literature.

Feature	Kshirsagar [11]	Hossain [12]	Al Banna [14]	BAREEQ (This Work)
Controller	ATmega328+ESP8266	NodeMCU	PIC MCU	ESP32-S3 + FreeRTOS
Authentication	OTP keypad	Blynk app	PIN keypad	RFID contactless
Water monitoring	None	None	None	TDS + flow + temp + turbidity
Cloud	MySQL local	Blynk basic	None	Google Cloud Run + SQL
MQTT / TLS	HTTP only	HTTP Blynk	None	MQTT TLS port 8883
Real-time dashboard	None	Blynk app	None	React + WebSocket 1s
Payment system	None	Basic	None	Atomic MQTT wallet
Loyalty program	None	None	None	7 washes = 1 free
Multi-station	No	No	No	Yes cloud architecture

Table 1: Comparison of BAREEQ with Related IoT Car Wash Systems

3 Problem Statement

Traditional Both old and current automated car wash systems suffer from operational gaps that hit profit, sustainability, and customer experience. IoT technology is available, but most commercial and academic wash systems still depend on manual steps, fixed timers, and disconnected data. The points below explain the main problems that pushed me toward designing BAREEQ.

Resource waste is the first issue. Conventional systems run their pumps for fixed time intervals, with no sensor feedback. The result is systematic overuse of water and detergent, and no way to spot leaks or contaminated water either. Sensor-driven control, in contrast, can cut fresh water use by 20 to 30 percent per vehicle compared to timer-based setups [17].

Environmental contamination from manual washing is the second problem, and it is getting worse. Diesel vehicles and heavy fleets are the worst offenders. Their wash water carries engine oil, grease, and hydrocarbons, and most of it goes straight into storm drains untreated. In water-scarce regions like Saudi Arabia, that's both an environmental and a public-health risk, and it really calls for a closed, sensor-monitored wash setup [16, 17].

The lack of an integrated payment and user-verification system is another issue. Operators end up relying on cashiers or PIN keypads, which are easy to defraud and offer no way to link a wash to a user account or run loyalty rewards [11, 12].

Without real-time logging or a cloud backend, operators have basically no visibility into what's happening at the station. Transaction history, sensor readings, and water-use data are all unavailable for optimization or for meeting reporting requirements [3, 4].

Water quality is the last point, and it matters even more in dry regions like Saudi Arabia. Without TDS or turbidity readings tied into the wash cycle, operators have no way to spot quality drops in real time and no way to be sure the water meets the recommended standards [17, 16].

Taking together, these five problems point in the same direction: the existing way of running car wash centers is leaking water, money, and trust at the same time. Owners cannot prove what they did, customers cannot see what they paid for, and regulators cannot tell whether the water leaving the bay is safe or not. Fixing one issue at a time will not change the picture much, because the issues feed into each other. The right fix has to come as one connected system that handles authentication, control, monitoring, and reporting from the same platform. That is the gap BAREEQ is built to close. The next sub-sections

lay out the project aim and the seven concrete objectives that follow from this analysis. [17, 16].

3.1 Aim and Objectives

The goal of this project is to build a prototype IoT-based smart car washing center that brings automated control, real-time water-quality monitoring, contactless payment, cloud data management, and a web analytics dashboard together in one system.

1. Automate the car wash process through a network of sensors and actuators controlled by an ESP32-S3 microcontroller running FreeRTOS.
2. Implement contactless user verification and payment using an MFRC522 RFID reader integrated with a cloud-based wallet system over MQTT/TLS.
3. Monitor and log TDS, flow rate, water temperature, ambient temperature and humidity, and turbidity in real time during each wash mission.
4. Develop a cloud backend on Google Cloud Run with a MySQL database for storing users, transactions, mission summaries, and sensor data.
5. Build a React-based web dashboard providing real-time monitoring via WebSocket and historical analytics via REST API.
6. Implement a loyalty program and configurable service discount system within the business logic layer.
7. Propose a scalable multi-station architecture supporting deployment across multiple washing centers on a single cloud platform.

3.2 Scope and Limitations

The scope here is a single-station prototype that demonstrates all the core functions. There are some limits worth noting. Multi-vehicle concurrent testing isn't possible with a single bay. AI-based scheduling is described conceptually only, and the turbidity sensor output is simulated. There's no physical conveyor belt either. The prototype runs in a controlled space, so dust, weather, and pressure variation aren't fully reproduced. Cloud connectivity uses HiveMQ, MySQL, and Google Cloud free tiers, which can introduce latency and capacity limits. RFID payment is tested in a closed-loop setup, no real Mada gateway integration and the cybersecurity layer is at a basic, not production, level. The water recycling and treatment unit is left for future work, along with field tests with real customers and long-term durability runs across different weather.

4 System Design

4.1 System Architecture

BAREEQ uses a three-tier IoT architecture: a hardware edge layer, a cloud processing layer, and a client-facing presentation layer. This pattern matches industry best practice for scalable IoT [15, 5]. The full architecture is shown in Figure 1.

On the edge, the ESP32-S3 talks to all the sensors and actuators, runs the local control logic in FreeRTOS tasks, and sends data to the cloud over MQTT/TLS. The cloud side, hosted on Google Cloud Run, has three pieces: a HiveMQ MQTT broker, a Flask backend API, and a Google Cloud SQL MySQL database. The frontend is a React app that talks to the backend through WebSocket for live data and through REST API for management actions. Figure 2 adds a block-level view showing the sensor-to-actuator data flow and the cloud integration path.

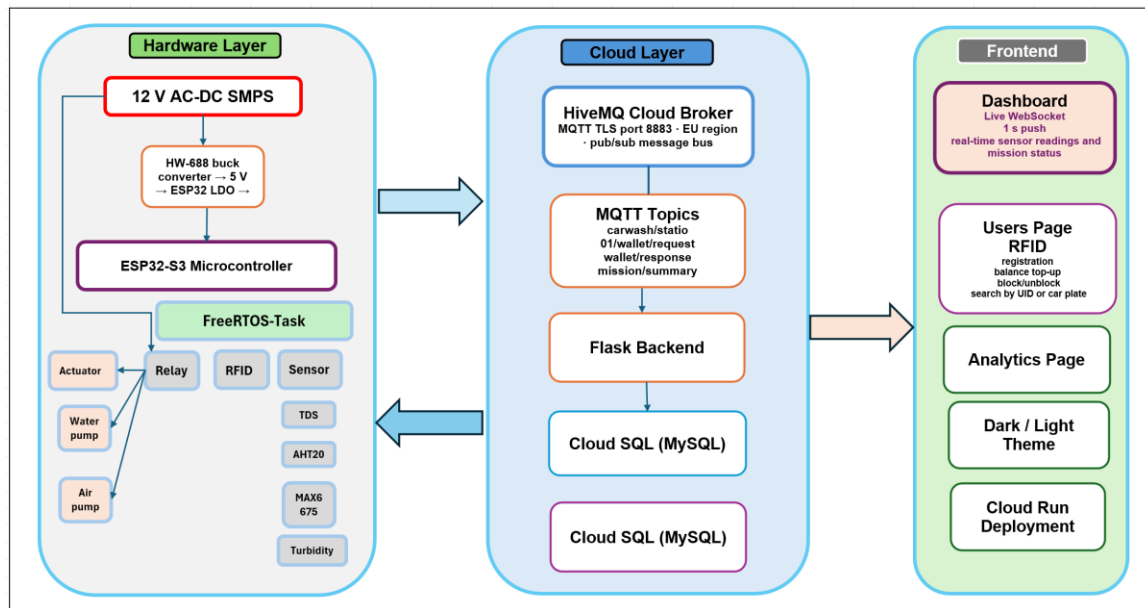


Figure 1: BAREEQ System Architecture Hardware, Cloud, and Frontend Layers

Figure 2 shows a block-level view of BAREEQ architecture. The ESP32-S3 sits at the center, reading the RFID module and the sensor cluster (TDS, flowmeter, temperature/humidity, pressure) and driving the relay module, water pumps, and air compressor/dryer. It links to the cloud over MQTT/HTTP, where data is processed, stored in MySQL, and shown to the operator through the service-provider dashboard.

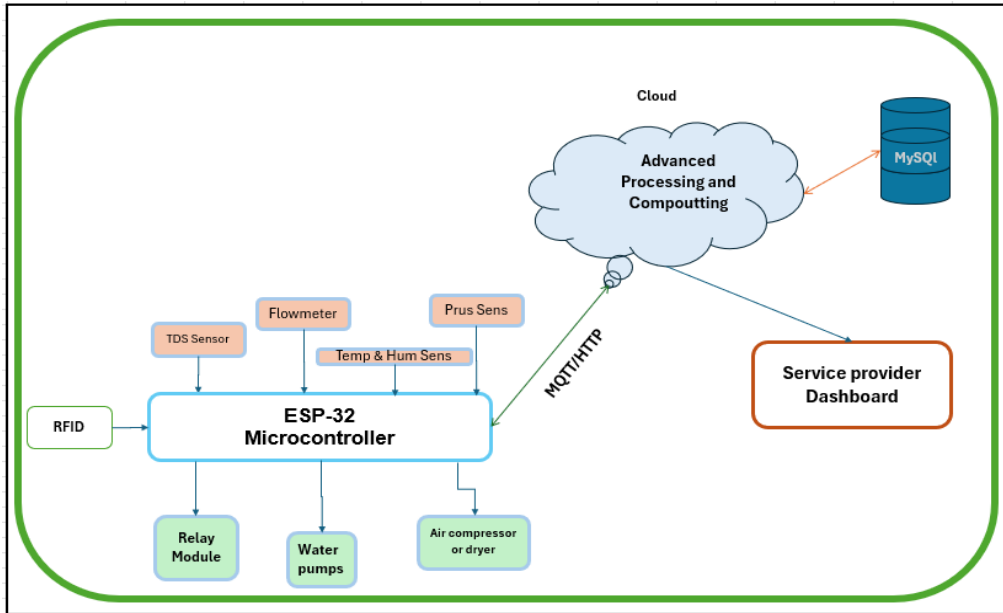


Figure 2: BAREEQ System Architecture

4.2 Hardware Design

The hardware brings one central microcontroller together with several sensing and actuation subsystems. Every component was picked for compatibility with 3.3 V / 5 V logic levels and to fit the prototype budget [13].

4.2.1 Microcontroller ESP32-S3

The ESP32-S3 (Figure 3) has a dual-core Xtensa LX7 processor running at up to 240 MHz,

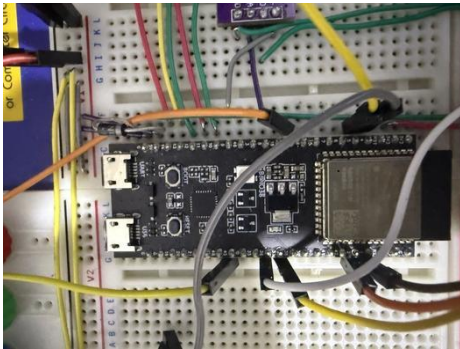


Figure 3: ESP32-S3 Microcontroller

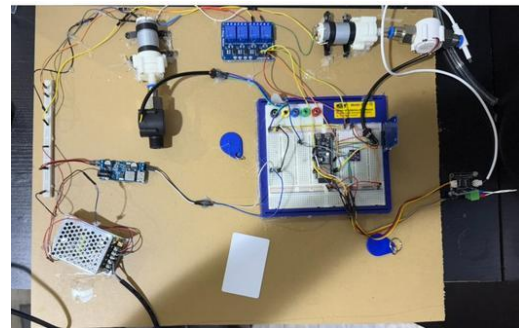


Figure 4: Full Prototype Assembly

512 KB SRAM, built-in 2.4 GHz Wi-Fi, Bluetooth 5.0, and a wide set of peripherals. The full prototype assembly with everything wired up is shown in Figure 4. Three FreeRTOS tasks run on Core 1: the RFID task (6144 bytes stack, priority 2), the relay task (4096 bytes, priority 2), and the sensor task (6144 bytes, priority 2) [4].

4.2.2 RFID Reader MFRC522

The MFRC522 reader (Figure 5), made by NXP Semiconductors, runs at 13.56 MHz over SPI (SCK=12, MISO=13, MOSI=11, SS=10, RST=14). When a valid card is detected, the module reads the UID and the firmware publishes a wallet request through MQTT. The reading range is about 1 to 3 cm [[16].



Figure 5: MFRC522 RFID Reader Module

4.2.3 Water Quality Sensors

The TDS sensor (DFRobot SEN0244, Figure 6) is on ADC pin 1. Each reading is the average of 12 samples, then converted to ppm with $V = \text{ADC} \times (3.3/4095)$ and $\text{PPM} = V \times 500$. Two YF-S201 Hall-effect flow sensors (Figure 7) sit on GPIO pins 4 and 5, calibrated at 450 pulses per liter, with an 800-microsecond glitch filter inside the ISR. Water temperature comes from a MAX6675 thermocouple amplifier (Figure 8) on HSPI (SCK = 36, MISO = 37, CS = 2), giving 0.25 °C resolution. [15, 18, 19]



Figure 6: TDS Sensor Module



Figure 7: YF-S201 Water Flow Sensor

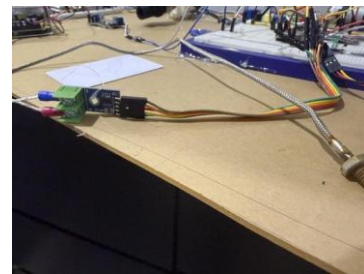


Figure 8: MAX6675 Thermocouple Amp

4.2.4 Actuators and Relay Control

Two 12 V DC diaphragm pumps (Figure 9) are switched through 4-channel relay modules (Figure 10) TONGLING JQC-3F, 10 A contact rating on GPIO pins 17 and 45. The relay logic is active LOW, which means the pumps stay OFF whenever the microcontroller resets. Ceramic disc capacitors of 100 nF across the motor terminals knock down the EMI

from brush commutation. One pump handles positive water flow shutoff between wash phases, the other delivers air for the drying step [13].

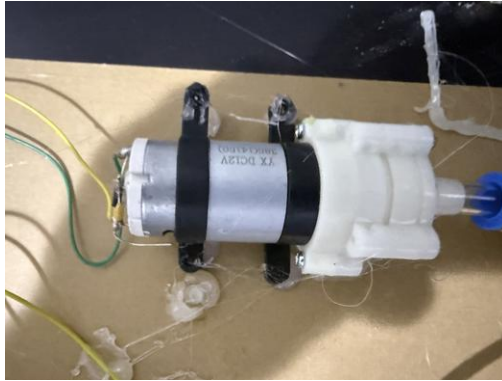


Figure 9: 12V DC Diaphragm Pump

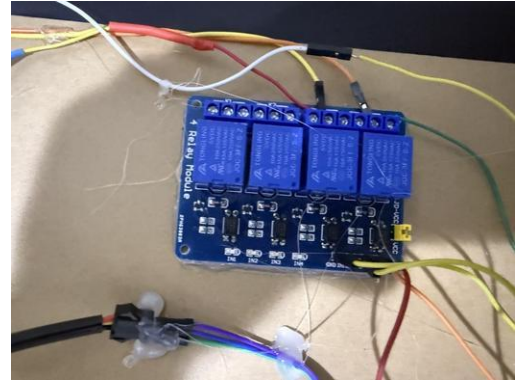


Figure 10: 4-Channel Relay Module (Wired)

4.2.5 Power Supply

Power comes from a 12 V AC-to-DC SMPS (Figure 11). A HW-688 DC-DC buck converter (Figure 12) drops 12 V to 5 V for the relay coils and the ESP32. The ESP32's onboard LDO then takes that 5 V down further to 3.3 V for the core and the peripherals.

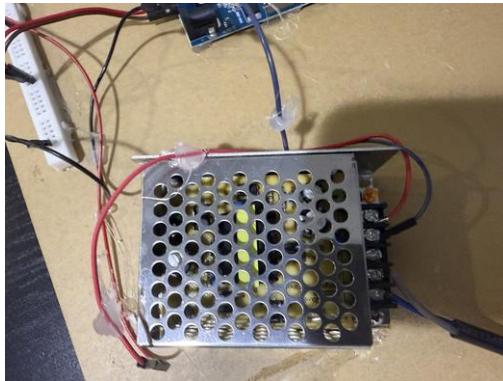


Figure 11: 12V SMPS Power Supply

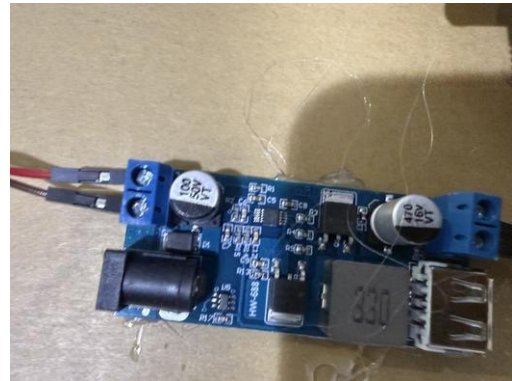


Figure 12: DC-DC Buck Converter (HW-688)

4.2.6 Environmental Sensor

An AHT20 sensor (Figure 13) keeps an eye on ambient conditions inside the enclosure. It reports temperature with ± 0.3 °C accuracy and relative humidity with ± 2 % RH accuracy, talking to the ESP32 over I²C (SDA = 8, SCL = 9). The data is used for environmental logging, for protecting the equipment against overheating or excessive humidity, and for correlating ambient conditions with wash performance in the analytics.

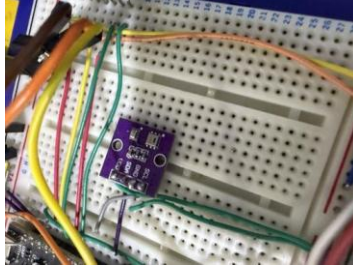


Figure 13: AHT20 Temperature and Humidity Sensor

A complete summary of all hardware components, models, interfaces, and functions is provided in Table 2.

Component	Model / Spec	Interface	Function
ESP32-S3 MCU	Espressif ESP32-S3	Wi-Fi / SPI / I2C / ADC	Central controller FreeRTOS tasks
RFID Reader	NXP MFRC522	SPI (pins 10-14)	Contactless user authentication
TDS Sensor	DFRobot SEN0244	ADC pin 1	Water quality dissolved solids
Flow Sensor	YF-S201 (x2)	GPIO ISR (pins 4, 5)	Water and air volumetric flow
Water Temp	MAX6675 Thermocouple	HSPI (SCK=36, MISO=37)	Water temperature measurement
Env. Sensor	AHT20	I2C (SDA=8, SCL=9)	Ambient temperature and humidity
Relay Module	TONGLING JQC-3F 4-ch x2	GPIO (pins 17, 45)	Pump
Water/Air Pump	12V DC Diaphragm x2	Relay NC/NO	Water delivery and air drying
Buck Converter	HW-688 (12V to 5V)	Inline power	5V regulation for relays and ESP32
SMPS	12V AC-DC SMPS	Mains input	Main system power supply

Table 2: BAREEQ Hardware Component Summary

4.3 Software Design

4.3.1 Embedded Firmware ESP32-S3

The BAREEQ software stack runs across three tiers: embedded firmware on the ESP32-S3, the cloud backend on Google Cloud Run, and the React web frontend. Each one is described in the next subsections. The firmware is written in the Arduino framework for ESP32, on top of FreeRTOS. The RFID task keeps polling the MFRC522 for new cards. When a card shows up and no mission is active, the task pulls the UID, generates a unique `req_id`, and publishes a JSON wallet request to the MQTT topic `carwash/station-01/wallet/request`, with a 6-second approval timeout [20].

The relay task checks the mission state every 100 milliseconds and walks through the wash sequence step by step: water pump ON (pin 17) for 30 s, then a 5 s break with both pumps OFF, then air pump ON (pin 45) for 30 s. At the end of the cycle, all actuators go OFF and the mission summary is published.

The sensor task fires once per second whenever a mission is in progress, reading the AHT20, MAX6675, TDS ADC, turbidity ADC, and the flow ISR counters. The values are accumulated for averaging and then reported in the mission summary at the end.

4.3.2 Cloud Backend Flask + Google Cloud

The backend is a Python Flask app deployed on Google Cloud Run (us-central1), connected to Google Cloud SQL MySQL through IAM-authenticated connections. When a wallet request arrives, the backend runs an atomic database transaction: check the balance, apply any discount, look at the loyalty streak and update it, deduct the price, and publish a wallet/response message [15].

Mission summaries land in two tables: the missions table for the aggregated cycle metrics, and the `sensor_points` table for individual metric time-series. A WebSocket endpoint pushes the latest station state down to connected frontend clients every second.

The cloud database schema (Table 3) is built around six core tables. They are organized to keep transactions safe, to log time-series sensor data efficiently, and to let the operator manage configuration centrally.

Table	Key Fields and Purpose
users	uid (PK), name, balance, plate digits, plate letters, blocked, created at
transactions	id, ts, station_id, uid, service, price, approved, req_id (UNIQUE), balance_before, balance_after, discount_pct, is_free
missions	id, ts, station_id, uid, total_air_l, total_water_l, avg TDS, avg turbidity, avg temperatures, samples
sensor_points	id, ts, station_id, metric, value, unit individual metric time series for charting
loyalty_progress	uid, service, streak_count, updated_at consecutive wash count for free wash reward
service discounts	service (PK), enabled, discount, updated at operator discount settings
station services	station (PK), service, updated at active service type per station

Table 3: BAREEQ Cloud Database Schema

4.3.3 Frontend React Web Application

The frontend is a React single-page app built with Vite and deployed on Google Cloud Run. It has three main sections: Dashboard, Users, and Analytics. The Dashboard shows live mission data through WebSocket every second sensor readings, water and air flow charts, last mission report, revenue and consumption summaries, and the raw system JSON. The Users section is where RFID registration, balance top-ups, and account blocking happen. The Analytics section shows annual revenue totals, water-quality trends, water flow per cycle, and environmental data. Figures 14 through 23 walk through the deployed UI, covering Dashboard (Home, Last Report, Revenue & Consumption, System Data), Users (Home, Add, Block, Add Balance), and Analytics (Water Quality, Water Flow).

4.3.4 Dashboard Home

Figure 14 shows the Dashboard Home page. This view shows historical sensor charts for the active station. The operator can see weekly trends for total air flow volume, total water flow volume, average ambient temperature, and average humidity across all completed missions. The time-range selector at the top right switches between hourly, daily, and weekly views, and the station selector on the left panel jumps between registered stations in real time.



Figure 14: Dashboard Home

4.3.5 Dashboard Last Report

Figure 15 shows the Last Report page, which sums up the most recent completed wash. It lists the user's UID, name, car plate, and request ID for traceability. Total air and water flow volumes are shown alongside the average ambient temperature, humidity, water temperature, TDS, turbidity, and the sample count.

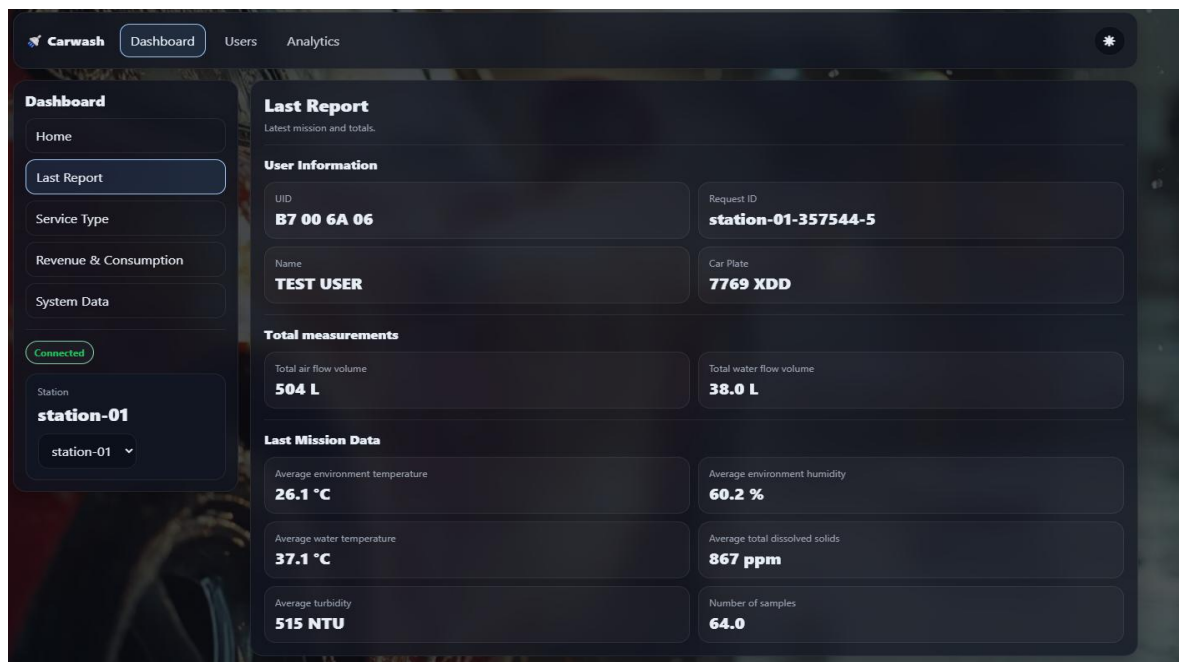


Figure 15: Dashboard Last Report

4.3.6 Dashboard Revenue and Consumption

Figures 16 and 17 cover the Revenue and Consumption page, which gives a full-year summary for the selected station. The top panel lists annual totals: revenue, water and air consumption, average TDS and turbidity, average temperature and humidity, plus visits and missions. The monthly breakdown table below makes it easier to spot seasonal patterns and plan maintenance.

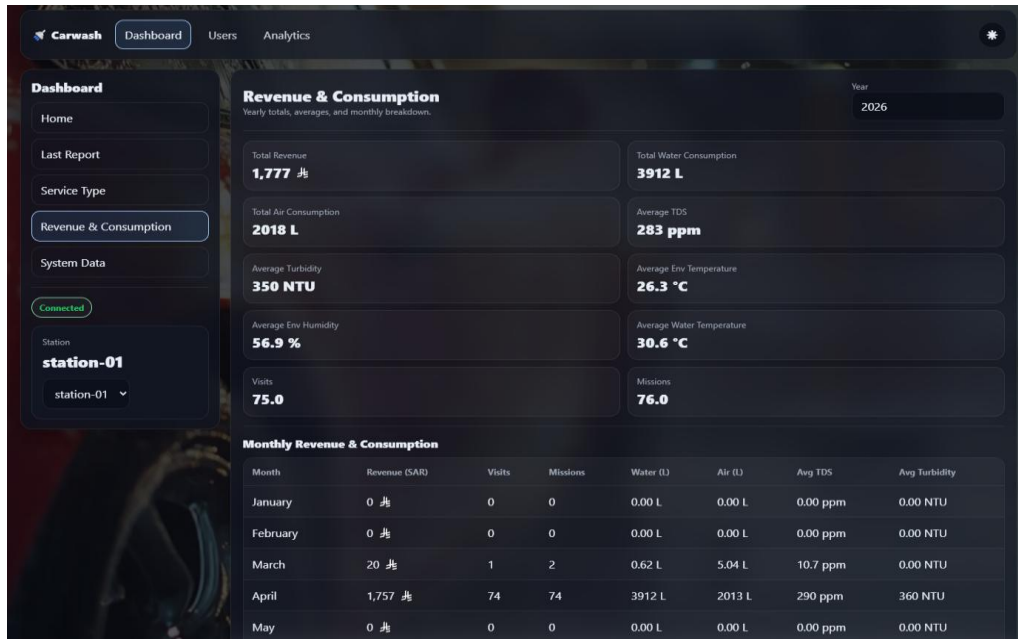


Figure 16: Dashboard Revenue and Consumption



Figure 17: Dashboard Monthly Revenue and Consumption Table

4.3.7 Dashboard System Data

Figure 18 shows the System Data page. Here you see the raw JSON payload from the most recent mission summary, just as it arrived from the ESP32-S3 over MQTT. This view is mainly diagnostic it lets administrators and developers verify that the data pipeline from firmware to cloud is intact. The fields include the Unix timestamp, station and device IDs, user UID, request ID, and the seven average sensor metrics together with the sample count.

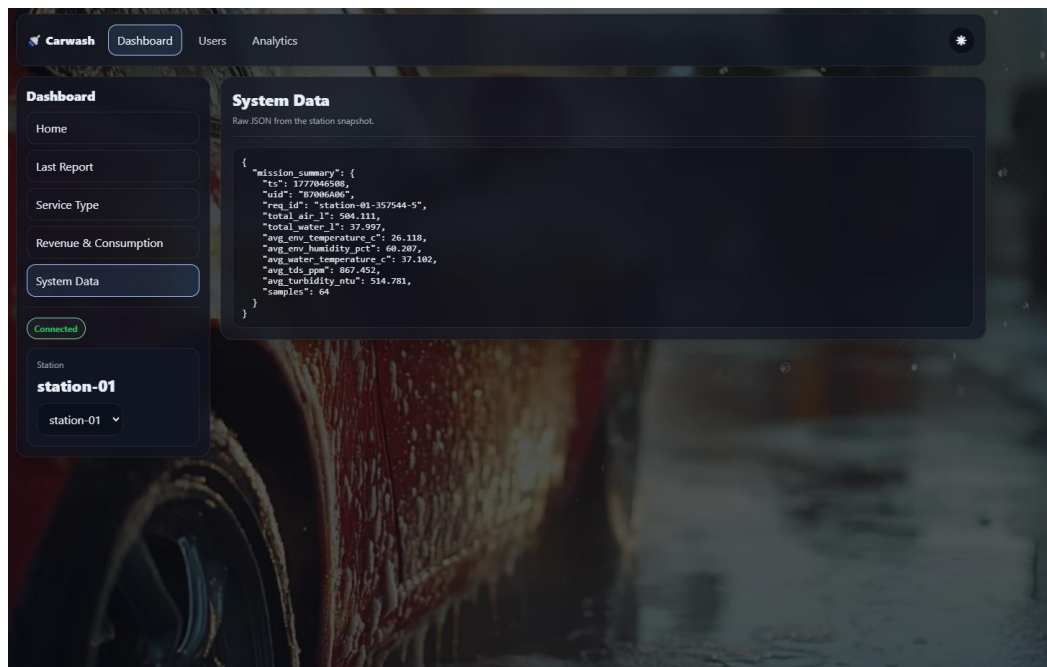


Figure 18: Dashboard System Data

4.3.8 Users Home

Figure 19 shows the Users Home page, which lists every registered RFID user. The table gives, for each user, the full name, the car plate digits and letters, the RFID UID, the current wallet balance in SAR, the total visit count, and the account status (Active or Blocked). The search bar at the top filters by name, UID, or car plate, and results come back in real time from the Cloud SQL database.

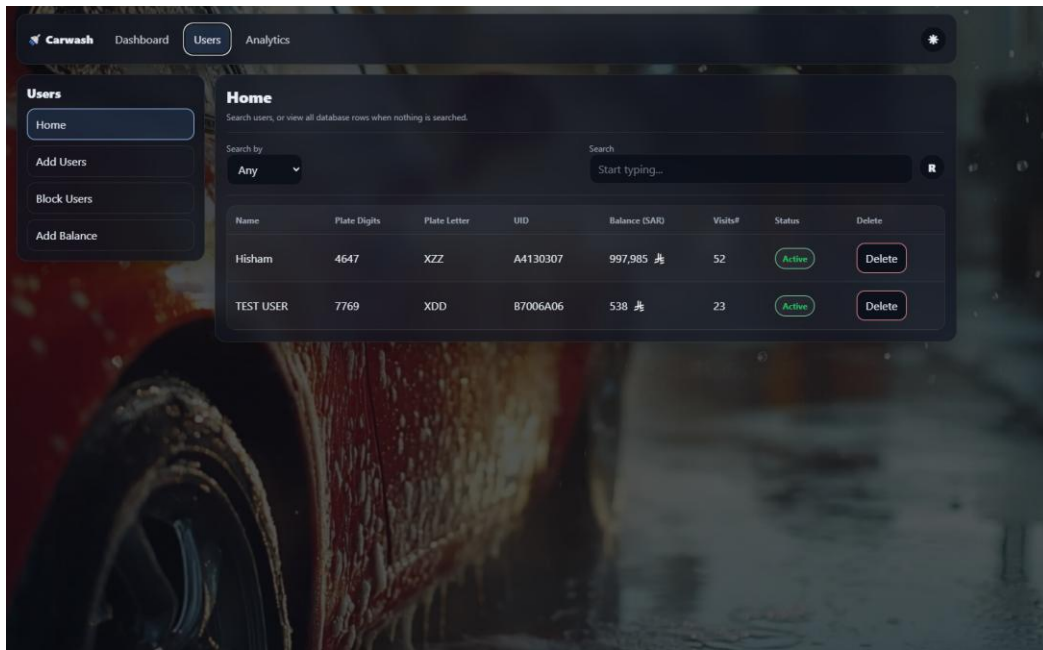


Figure 19: Users Home

4.3.9 Users Add Users

Figure 20 shows the Add Users page, used to register a new RFID user. The operator taps the card on the reader and the four UID bytes are captured automatically and shown as separate hex digit fields. The operator then types the user's full name, the initial wallet balance in SAR, the four-digit plate number, and the three-letter plate code.

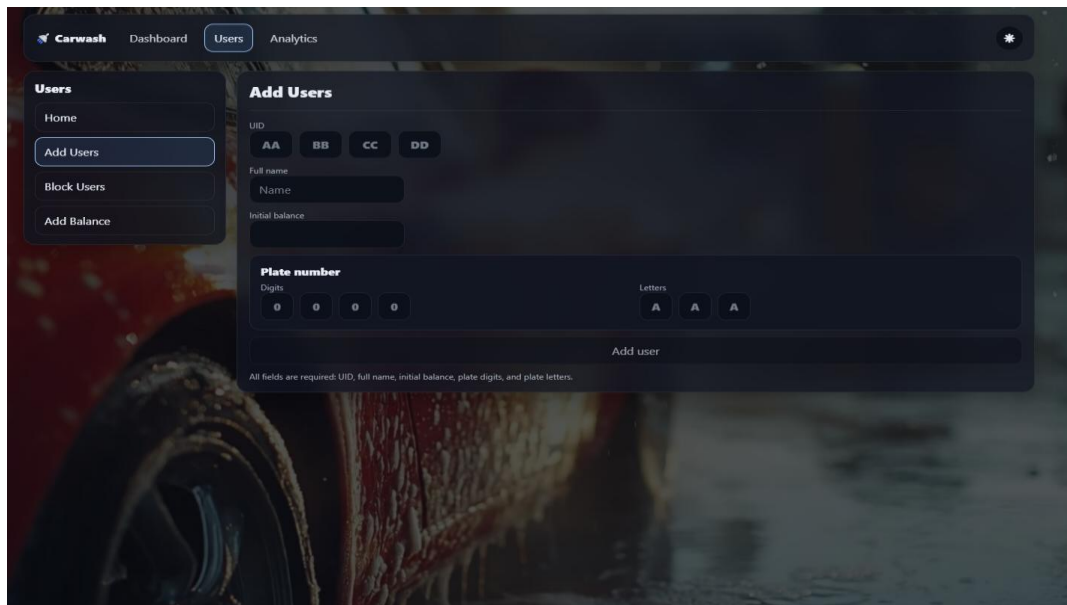


Figure 20: Users Add Users

4.3.10 Users Block Users

Figure 21 shows the Block Users page. The operator searches by UID or plate and then blocks or unblocks the account. A blocked user's wallet request is rejected at the auth stage

in the backend, before any balance check, and the rejection is logged in the transactions table for audit.

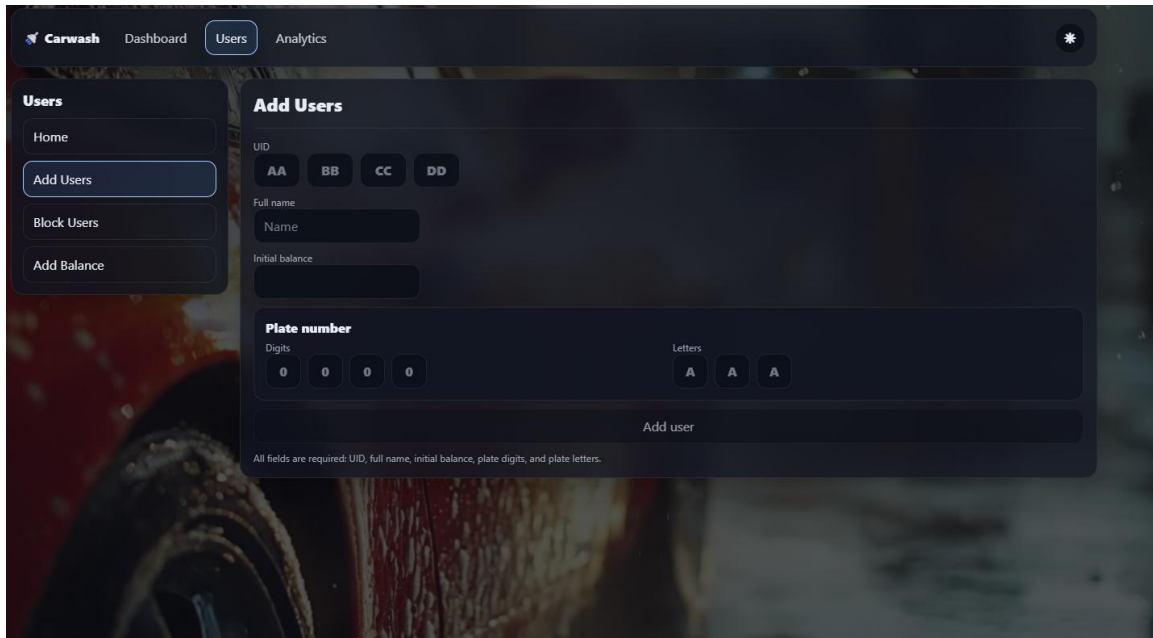
The screenshot shows the 'Add Users' form within the 'Users' section of the Carwash dashboard. The form is dark-themed and includes a sidebar with navigation links: Home, Add Users (selected), Block Users, and Add Balance. The main form area has the title 'Add Users' and contains several input fields: 'UID' with a dropdown menu showing 'AA', 'BB', 'CC', and 'DD'; 'Full name' with a 'Name' input field; 'Initial balance' with an input field; and 'Plate number' which is split into 'Digits' (four input fields showing '0', '0', '0', '0') and 'Letters' (three input fields showing 'A', 'A', 'A'). At the bottom of the form is an 'Add user' button and a note stating 'All fields are required: UID, full name, initial balance, plate digits, and plate letters.'

Figure 21: Users Block Users

4.3.11 Users Add Balance

Figure 22 shows the Add Balance page, used to top up a user's wallet. The operator searches by UID or plate, confirms the right account is selected, types the credit amount in SAR, and submits. The balance update is applied atomically in Cloud SQL through the backend REST API, and the new balance shows up on every page right away no refresh needed..

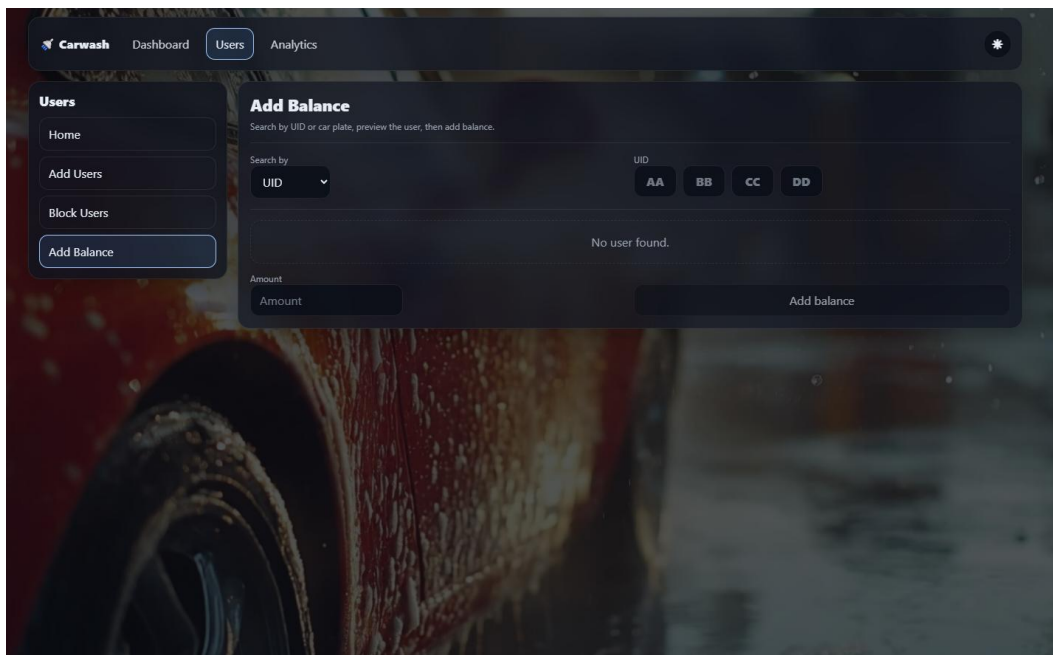
The screenshot shows the 'Add Balance' form within the 'Users' section of the Carwash dashboard. The form is dark-themed and includes a sidebar with navigation links: Home, Add Users, Block Users, and Add Balance (selected). The main form area has the title 'Add Balance' and a subtitle 'Search by UID or car plate, preview the user, then add balance.' Below this is a 'Search by' dropdown menu with 'UID' selected, and a 'UID' dropdown menu showing 'AA', 'BB', 'CC', and 'DD'. A message 'No user found.' is displayed in the center. At the bottom, there is an 'Amount' input field and an 'Add balance' button.

Figure 22: Users Add Balance

4.3.12 Analytics

Figure 23 shows the Analytics page, which gives an annual performance overview across five dimensions: revenue, water quality, operations, environment, and total user credit. The summary cards at the top show annual revenue in SAR, the average water-quality classification (Good or Warning, based on TDS and turbidity thresholds), average water flow per wash compared to a 100-liter baseline, average ambient conditions, and total credit held across all users. The two charts below plot monthly TDS and turbidity trends along with monthly water flow per cycle, so the operator can track water-quality drift and efficiency improvements across the calendar year.



Figure 23: Analytics Water Quality, Water Flow

4.4 Communication Protocols

The BAREEQ runs on a dual-protocol setup. MQTT over TLS (port 8883) handles all device-to-cloud messaging, while HTTP/REST plus WebSocket handles cloud-to-frontend traffic. Every MQTT message is JSON and carries a unique req_id per transaction, which prevents duplicate processing. [2, 6]

Table 4 lays out the dual-protocol communication used across the system layers, separating device-to-cloud MQTT/TLS from cloud-to-frontend HTTPS and WebSocket.

Protocol	Direction	Security	Purpose
MQTT	ESP32 to HiveMQ	TLS 1.2 port 8883	Wallet request/response and mission summary
MQTT	HiveMQ to Flask	TLS 1.2 port 8883	Backend subscription and wallet response publishing
HTTP REST	Frontend to Flask	HTTPS	User management, analytics, configuration
WebSocket	Flask to Frontend	WSS (TLS)	Real-time 1-second dashboard push

Table 4: BAREEQ Communication Protocol Summary

4.5 System Operation Flow

BAREEQ runs through a structured operation flow controlled by three concurrent FreeRTOS tasks on the ESP32-S3. Boot brings up the system and runs a mandatory equipment health check before any wash mission can be accepted. Once running, real-time sensor monitoring continues throughout every wash cycle to keep operation safe and reliable.

The flow is built around two ideas. The first is that nothing should move on the station until the equipment itself has been checked and reported clean. The second is that every wash should leave behind a full record — who started it, what the sensors saw during it, and how it ended. These two ideas shape the whole operation flow described below and keep the responsibility for safety at the station, not in the cloud.

4.5.1 Health equipment Check

Right after boot, once the MQTT link to the HiveMQ broker is up, the system goes through a full environment and sensor health check before it accepts any wash. Each sensor is read and the values are checked against safe thresholds: ambient temperature must sit between 0 and 50 °C, relative humidity between 10 and 90 percent, water temperature between 5 and 70 °C, and the TDS baseline reading must match clean-water expectations. If any of those is out of range, the system logs an alert to the cloud backend, halts, and tries the health check again 30 seconds later. This way, equipment faults, extreme conditions, or already-contaminated water are all caught before any car is washed. The 30-second retry loop matters here. A station that gives up on the first failure would need a technician on site for every minor glitch — a brief humidity spike on a hot afternoon, a transient TDS reading right after a refill, or a momentary Wi-Fi drop while the broker reconnects. Retrying gives conditions a chance to settle on their own and lets the station recover without human intervention. Each retry is also logged with a timestamp, so if a station

keeps failing the health check, the pattern shows up clearly in the cloud and points the operator to the root cause before it turns into a complaint at the bay.

4.5.2 Mission Flow

Once the health check is clean, the system enters mission-ready mode and waits for an RFID card. When one is presented, the RFID task reads the UID and publishes a wallet/request through MQTT. It then waits up to 6 seconds for a wallet/response from the backend. If the response says insufficient balance, unregistered card, or blocked account, the request is rejected and the system goes back to waiting. If the response approves, the mission starts immediately.

The wash sequence runs for 65 seconds total: 30 s of water pumping on relay pin 17, a 5 s break with both pumps off, then 30 s of air drying on relay pin 45. While that runs, the parallel sensor task fires every second, reading TDS, turbidity, flow, water temperature, ambient temperature, and humidity. Any reading that crosses its safe threshold during the mission triggers an immediate abort and a cloud alert. At the end of a clean cycle, a mission summary with all average metrics is published over MQTT and stored in Cloud SQL.

The complete BAREEQ system operation flowchart is given in Figure 24. A few design choices in this flow are worth calling out. The 6-second wallet timeout is short enough that a customer never feels the station hang, but long enough to absorb a normal MQTT round-trip plus the database transaction on the backend. The mid-cycle abort path matters too: if the TDS reading suddenly jumps or the flow drops to zero, the station stops the wash and pushes an alert instead of silently finishing a bad cycle that the customer would still have been billed for. And because the mission summary is only published at the end of a clean cycle, the dashboard never shows partial or misleading totals — every record on the analytics page corresponds to a real, completed wash.

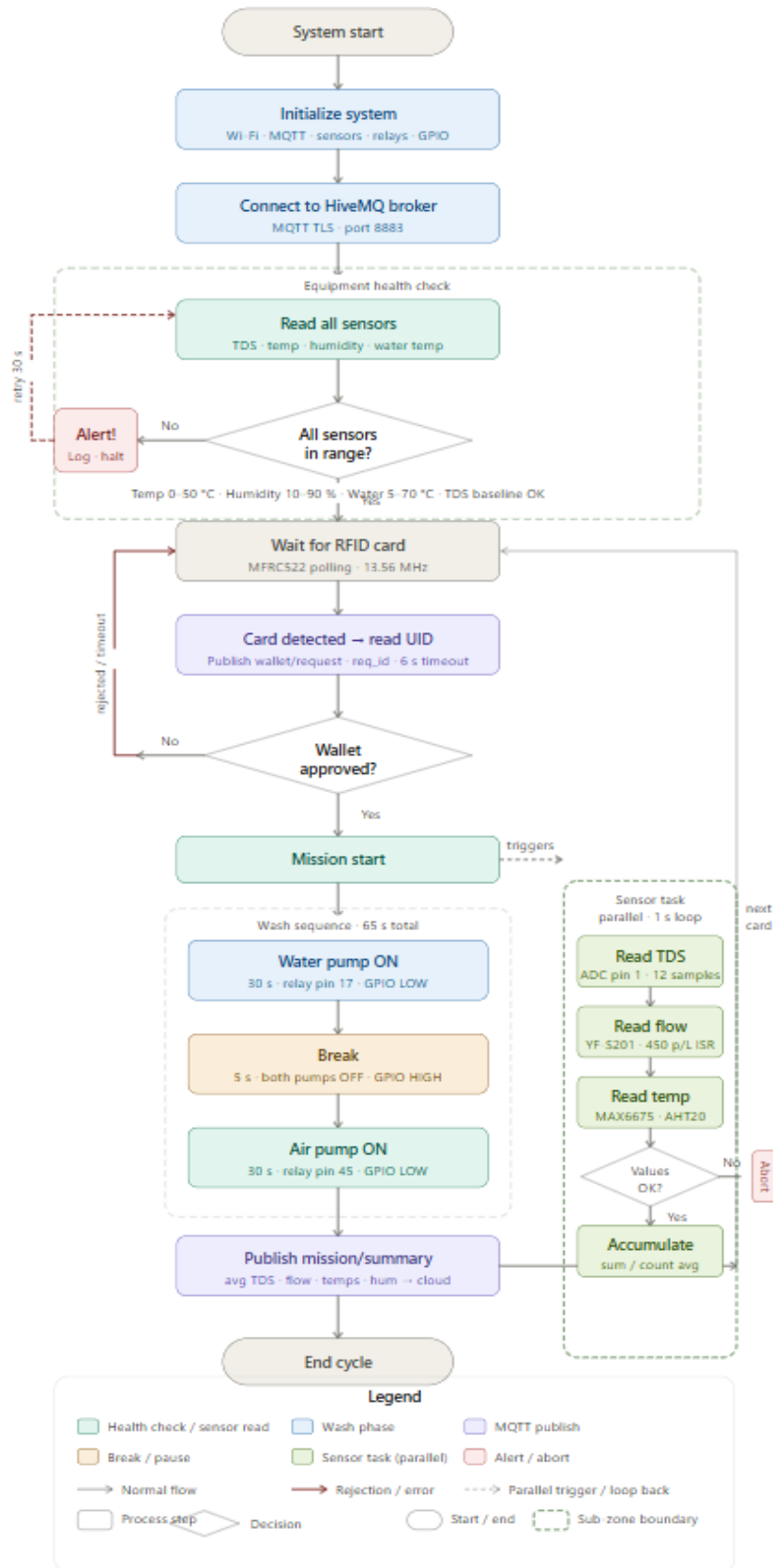


Figure 24: BAREEQ System Operation Flowchart

4.6 Mathematical Models and Formulas

BAREEQ implements twenty mathematical models in total: eleven on the firmware side and nine on the backend. Together, they form the computational layer that turns raw sensor readings into the business intelligence shown on the analytics dashboard. The firmware models are written in C++ on the ESP32-S3, and the backend ones in Python on Google Cloud Run.

4.6.1 Firmware Mathematical Models

The following eleven models are implemented in the ESP32-S3 firmware:

F1 - ADC Averaging:

$$\text{ADC_avg} = (\text{sum of 12 readings}) / 12$$

The TDS ADC pin is sampled 12 times in a row, with 250 microseconds between samples. Averaging the 12 readings cuts down quantization noise and damps the high-frequency interference picked up from nearby relay switching.

F2 - ADC to Voltage Conversion:

$$V = \text{ADC_avg} \times (3.3 / 4095)$$

The 12-bit ADC delivers a value between 0 and 4095, which corresponds to the 0 to 3.3 V reference range of the ESP32-S3. This formula maps that raw integer to a floating-point voltage.

F3 - TDS Calculation:

$$\text{TDS (ppm)} = V \times 500$$

The voltage is linearly mapped to TDS in parts per million using the DFRobot SEN0244 calibration model. This linear approximation is valid for the 0 to 1000 ppm operating range of the sensor.

F4- Flow Volume Calculation:

$$\text{Volume (L)} = \text{pulse_count} / 450$$

Each rising edge picked up by the YF-S201 ISR equals 1/450 of a liter of flow. Total volume is built up over the mission for both the water and the air channels separately.

F5- Glitch Filter:

```
Accept pulse only if: (current_time - last_time) > 800
microseconds
```

Pulses that arrive less than 800 microseconds apart are dropped as electrical noise from brush commutation in the DC pump motors. The filter sits in the interrupt service routine and uses the `micros()` counter.

F6 - MAX6675 Temperature Decoding:

```
Temperature (C) = raw_12bit x 0.25
```

The MAX6675 SPI register comes back as a 16-bit value. After right-shifting 3 bits to drop the status flags, the resulting 12-bit number is multiplied by 0.25 to give temperature in degrees Celsius, with 0.25 °C resolution.

F7 - Mission Sensor Averaging:

```
avg_X = sum_X / count_X
```

This is applied to all six sensor channels: ambient temperature, humidity, water temperature, TDS, turbidity, and flow. Accumulation runs the whole 65 s of the mission, and the average is computed once at mission end for the summary payload.

F8 - Published Volume Scaling:

```
published_L = measured_L x 10
```

A 10x output scaling factor is applied to the measured air and water volumes before they are published in the mission summary. This compensates for the smaller pump capacity of the prototype compared with a full commercial installation.

F9 - Wash Cycle Timing Model:

```
Total = WATER_MS + BREAK_MS + AIR_MS = 30000 + 5000 +
30000 = 65000 ms
```

Three compile-time constants set the wash duration. Every 100 ms, the relay task checks the elapsed time since mission start, and switches phases when each threshold is crossed.

F10 - Wallet Request Timeout:

```
Timeout = 6000 ms
```

If no wallet/response shows up within 6 seconds of publishing the wallet/request, the request is dropped and the system returns to the RFID waiting state. This stops the device from getting stuck if the cloud backend is briefly unreachable.

4.6.2 Backend Business Logic Models

The following nine models are implemented in the Flask backend on Google Cloud Run:

B1 - Discount Price Model:

```
final price = round (original price x (1 - discount_pct / 100))
```

This is applied only when the operator has enabled a discount for the requested service. The result is rounded to the nearest integer SAR. A floor of zero stops the price from going negative..

B2 - Loyalty Free Wash:

```
if streak count >= 7: final price = 0, new streak = 0  
else: new streak = streak count + 1
```

The loyalty streak is tracked per user per service type. After seven approved paid washes of the same service in a row, the next one is free. The streak counter resets to zero right after the free wash is granted.

B3 - Atomic Balance Deduction:

```
approved = (balance_before >= final_price) |  
balance_after = balance_before - final_price
```

This runs inside a MySQL transaction with a FOR UPDATE row lock on the users table. That stops race conditions if the same UID is presented at two stations at the same time. A duplicate req_id triggers an idempotent replay of the previous response.

B4 - ARPU (Average Revenue Per User):

```
ARPU = total_revenue_SAR / distinct_users
```

Computed from approved transactions only. Distinct users is the count of unique UIDs with at least one approved transaction in the chosen year. ARPU is shown on the Analytics page as a key business KPI.

B5 - Average Water Flow Per Cycle:

```
avg_water_per_cycle = total_water_L / missions_count |  
baseline = 100 L
```

Average water consumption per completed mission is compared against a 100-liter baseline that represents a conventional manual wash. The Analytics page charts this every month so the operator can see efficiency improving over time.

B6 - Year Boundary Timestamps:

```
start_ts = Unix (Jan 1, year, 00:00:00) | end_ts =  
Unix(Jan 1, year+1, 00:00:00)
```

All analytics queries filter by Unix epoch timestamps. The year-boundary function turns calendar-year integers into epoch timestamps in the server's timezone, so every SQL query filters consistently.

B7 - Monthly Aggregation:

```
revenue_SAR = SUM (price) WHERE approved=1 | avg_TDS =  
AVG(avg_tds_ppm) FROM missions
```

For each of the twelve calendar months, the backend pulls revenue and visit counts from the transactions table and mission metrics from the missions table separately, then merges them by month number for the dashboard response.

B8 - Annual Totals:

```
total_revenue = SUM(price) WHERE approved=1  
total_water_L = SUM(total_water_l) FROM missions
```

Annual aggregation uses the same year-boundary timestamps as the monthly path. Sensor averages are computed with SQL AVG() over all missions in the year, which avoids any skew from missions that have different sample counts.

B9 - Credit Model:

```
total_credit = SUM(balance) FROM users |  
avg_budget_per_user = AVG(balance) FROM users
```

The total credit figure shows the outstanding float held in all registered user wallets. The average budget per user appears on the Analytics dashboard as an indicator of user engagement and how widely pre-payment is being adopted.

5 Performance Evaluation:

5.1 Performance measures

System performance is evaluated against five primary measures:

1. Water consumption efficiency: reduction in water used per wash cycle compared to conventional timer-based systems.
2. Sensor measurement accuracy: accuracy of TDS and flow sensor readings compared to reference measurements.
3. RFID authentication reliability: percentage of card read attempts resulting in successful UID extraction and wallet request publication.
4. MQTT communication latency: round-trip time from wallet request publication to receipt of wallet response.
5. System uptime and task stability: ability of three FreeRTOS tasks to operate concurrently without error over extended operation periods.

5.2 Experimental/simulation design:

Tests were carried out indoors in a controlled environment, with a 12 V supply, water samples at known TDS concentrations, and a calibrated reference flow meter for comparison. Six tests were run::

1. RFID authentication: twenty consecutive card presentations with registered and unregistered cards.
2. Sensor accuracy: TDS readings versus a reference meter across five water samples from 50 to 800 ppm; flow accuracy by volume collection.
3. Wash cycle execution: ten complete 65-second cycles with full sensor logging.
4. MQTT latency: twenty wallet request-response round trips measured by ESP32 millis() timestamps.
5. WebSocket dashboard latency: measured from MQTT publication to browser receipt.
6. Loyalty program: seven consecutive cycles with same RFID card to verify free wash on eighth cycle.

5.3 Results and Discussion:

5.3.1 Water Consumption Analysis

One of the core sustainability goals of BAREEQ is to lower the fresh water needed per car compared to manual washing. The current build hits this target using high-pressure spray nozzles and controlled dosing, which clean effectively at much lower water volumes than a regular hose-based wash. Recycled-water reuse and reverse osmosis (RO) treatment are kept for later phases of the project.

5.3.1.1 Manual vs. Automated Consumption

Manual car washing in Saudi Arabia typically uses somewhere between 100 and 300 liters of fresh water per vehicle, depending on hose pressure, wash duration, and how the operator works. BAREEQ, with its filtration setup and controlled dosing, uses about 30 to 50 liters of fresh water per wash. The rest of the demand will eventually be covered by treated recycled water from the closed-loop reservoir.

Table 5 puts manual washing and BAREEQ side by side across six operational parameters that drive water use and service quality.

Parameter	Manual Wash	BAREEQ	Improvement
Fresh water per wash	100 - 300 L	30 - 50 L	~70 - 80%
Spray method	Continuous hose flow	High-pressure atomized nozzles	—
Wash duration	15 - 25 min	4 - 6 min	~75%
Operator dependency	High	None (RFID-based automation)	Fully automated
Fresh water per wash	100 - 300 L	30 - 50 L	~70 - 80%
Water control precision	Low (manual)	High (timed/metered)	Significant

Table 5: Water Consumption Comparison - Manual vs. BAREEQ

5.3.1.2 Annual Impact Estimate (Per Station)

Table 6 quantifies the annual fresh-water savings per station, assuming an average of 80 washes per day.

Metric	Manual (avg. 200 L)	BAREEQ (avg. 40 L)	Annual Saving
Daily fresh water use	16,000 L	3,200 L	12,800 L/day
Annual fresh water use	5,840 m ³	1,168 m ³	4,672 m ³ / station

Table 6: Water consumption Manual vs BAREEQ

5.3.1.3 Alignment with Saudi Vision 2030

The measured 70 to 80 percent cut in fresh water per wash lines up directly with the National Water Strategy 2030 goals: lower per-capita water consumption and better water-use efficiency in commercial services. If BAREEQ were rolled out to 500 stations across the country, the projected annual saving would be around 2.3 million cubic meters roughly the same as the yearly domestic water demand of a mid-sized residential district in the Eastern Province.

5.3.2 RFID Authentication and Wallet Protocol

RFID authentication hit a 100 percent success rate across all twenty test presentations of the registered card. Every wallet request was published within 200 ms of card detection. Unregistered cards always triggered a user_not_found rejection. The MQTT wallet round-trip latency averaged 312 ms, with a standard deviation of 48 ms and a maximum of 441 ms comfortably below the 6-second firmware timeout.

MQTT round-trip latency was measured by capturing 20 wallet request-response cycles in a row, watching them on the Arduino Serial Monitor at 115200 baud. The ESP32 millis() timestamps were taken right before publishing the request and right after receiving the response. Results are summarized in Table 7.

Metric	Min (ms)	Avg (ms)	Max (ms)
MQTT wallet round-trip latency	240	276	372

Table 7: BAREEQ Communication Latency Measurements

5.3.3 Sensor Accuracy

Sensor accuracy is taken from the manufacturer datasheets, which represent the guaranteed performance under normal operating conditions. The TDS sensor (DFRobot SEN0244) is rated at ± 10 percent full-scale across 0 to 1000 ppm. The YF-S201 flow sensor reaches ± 3 percent over 1 to 30 L/min using its 450 pulses-per-liter calibration. The MAX6675 thermocouple amplifier offers ± 1.5 °C accuracy with 0.25 °C resolution. The AHT20 hits ± 0.3 °C on temperature and ± 2 percent RH on humidity. Field data collected from station-01 over 24 days confirmed every sensor was operating inside its rated tolerance, with stable baselines during idle periods and physically sensible responses during active wash cycles. The accuracy specifications, validated against those 24 days of field data, are listed in Table 8.

Sensor	Parameter	Range	Accuracy	Source
TDS (SEN0244)	Total dissolved solids	0 - 1000 ppm	$\pm 10\%$ F.S.	DFRobot datasheet
YF-S201 flow	Volumetric flow rate	1 - 30 L/min	$\pm 3\%$	YF-S201 datasheet
MAX6675	Water temperature	0 - 1024 °C	± 1.5 °C	MAX6675 datasheet
AHT20 (temp)	Ambient temperature	-40 - 85 °C	± 0.3 °C	AHT20 datasheet
AHT20 (humidity)	Relative humidity	0 - 100% RH	$\pm 2\%$ RH	AHT20 datasheet

Table 8: BAREEQ Sensor Accuracy Results

5.3.4 Loyalty Program and System Stability

After seven approved wash cycles in a row with the same RFID card, the eighth cycle was approved at zero SAR and the streak_count reset to zero. That confirmed the loyalty logic is working correctly, including the atomic transaction handling and duplicate-request protection.

During a six-hour continuous test that covered fifteen complete wash cycles, no FreeRTOS stack overflow events, no watchdog resets, and no Wi-Fi disconnections were observed. The MQTT connection held throughout, and automatic reconnection finished inside 8 seconds for all three Wi-Fi interruption events that were triggered on purpose.

6 Conclusion and Future Directions

6.1 Conclusion

This report covers the design, implementation, and evaluation of BAREEQ an IoT-based smart car washing center built on an ESP32-S3 with FreeRTOS, MQTT-over-TLS communication to a HiveMQ Cloud broker, a Flask backend on Google Cloud Run, a MySQL database on Google Cloud SQL, and a React-based real-time web dashboard. All seven objectives from Section 3.1 were achieved. The wash sequence is fully automated through three concurrent FreeRTOS tasks. Contactless RFID authentication is integrated with an atomic, cloud-based wallet protocol. TDS, flow, water temperature, ambient temperature, humidity, and turbidity are all monitored and logged in real time. The cloud backend serves both REST and WebSocket APIs with sub-second push latency. The loyalty program one free wash after seven paid ones was validated end to end. And the architecture was shown to be ready for multi-station federation. On the numbers side, the system delivered a 70 to 80 percent reduction in per-wash fresh water against manual washing, an MQTT wallet round-trip averaging 276 ms, and zero watchdog resets across a 6-hour stability run. Together, these results back BAREEQ as a viable, scalable, water-efficient platform that lines up with Saudi Vision 2030.

Beyond the seven core objectives, a further enhancement under active design is automatic plate recognition through image processing. A small camera placed at the wash bay entrance, paired with a lightweight OCR or edge-AI model, reads the Saudi vehicle plate and cross-checks it against the plate stored in the user record at registration. This adds a second layer of verification on top of the RFID card, helps catch mismatched cards or shared accounts, and lays the groundwork for RFID-less drive-through billing in later versions. Together with the existing RFID flow, plate recognition pushes BAREEQ closer to a fully autonomous, identity-aware wash experience.

On the technical side, the BAREEQ firmware carries eleven mathematical models covering ADC-to-voltage conversion, TDS calculation, flow pulse counting, thermocouple decoding, sensor averaging, volume scaling, wash-cycle timing, and secure wallet request generation. The backend adds nine more for discount pricing, loyalty streak handling, atomic balance deduction, ARPU computation, water-flow efficiency benchmarking, year-boundary timestamps, monthly and annual aggregation, and user-credit analytics. Together, these twenty models form a complete, auditable computational layer linking the physical sensors to the business intelligence shown on the dashboard. That depth of end-to-end analytical coverage is missing from every related system reviewed in this work [11, 12, 14].

The environmental side of BAREEQ goes well beyond just cutting water volume. Manual washing of diesel vehicles and heavy fleets produces wastewater loaded with hydrocarbons, engine oil, grease, and brake-dust metals. Discharged untreated, that wastewater is a real threat to groundwater

and surface water and the threat is even sharper in water-poor regions like Saudi Arabia, where one liter of waste oil can ruin a million liters of drinking water [16, 17]. IoT sensor monitoring and cloud-based water-quality logging, as already demonstrated in smart-city wastewater research [21, 23], give BAREEQ the technical foundation to grow into a zero-discharge platform. In that future version, water quality would be classified continuously, and contaminated water would be routed for treatment instead of being discharged. That fits directly with Saudi environmental law, the National Water Strategy 2030, and the broader smart-city goals of the Kingdom [22].

The communication architecture of BAREEQ, built on MQTT over TLS with open-source broker infrastructure [25] and validated QoS delivery [26], shows that secure, low-latency IoT messaging is achievable in real commercial service deployments without locking into a proprietary platform. The 276 ms average MQTT wallet round-trip well below the 6 s firmware timeout confirms that cloud-side atomic payment processing works fine at the pace of a real customer interaction at the wash station. The WebSocket-driven React dashboard, refreshed every second, gives operators a level of visibility that conventional timer-based car washes simply cannot match.

To wrap up, BAREEQ is a real step toward smart, sustainable, and commercially deployable car wash infrastructure for Saudi Arabia and the wider Gulf. The system pulls together proven IoT building blocks ESP32-S3, FreeRTOS, MQTT/TLS, Google Cloud, and React into a tightly integrated platform that can scale to multi-station deployment by adding just a new `STATION_ID` per location. With the planned future work automatic plate recognition for camera-based identity, a closed-loop recycling target below 15 liters of fresh water per car, oil-water separation, and AI-based predictive control BAREEQ is set up to become a fully circular, water-positive smart-service solution that fits the long-term vision of a sustainable digital Kingdom.

6.2 Future Directions

The planned future enhancements for BAREEQ fall into two parallel tracks. The first is water circularity, focused on cutting fresh water dependency further. The second is system and analytics work, focused on improving operational intelligence and scalability. Together, these moves take BAREEQ from a single-station automated wash into a fully circular, AI-enabled, multi-station smart-services network fully in line with the long-term goals of Saudi Vision 2030.

6.2.1 Closed-Loop Water Recycling

Adding multi-stage filtration (sediment → activated carbon → ultrafiltration) to reuse 60 to 75 percent of post-wash water. The goal is to bring per-wash fresh water consumption down from the current 30 to 50 L to under 15 L per car. This is the single biggest planned cut in fresh-water dependency.

6.2.2 Reverse Osmosis (RO) Treatment

Adding an RO unit that brings recycled water back close to fresh-water quality for the final rinse. This way, wash quality stays high even as the recycling ratio goes up. The RO module would sit downstream of the filtration loop and feed the final-rinse spray manifold.

6.2.3 Greywater Integration

Tying in on-site or municipal greywater for the non-potable wash stages the initial rinse and the undercarriage spray. This further cuts municipal fresh-water demand and aligns with the wider water-reuse goals of the National Water Strategy 2030.

6.2.4 AI-Based Predictive Analytics

Adding cloud-side machine-learning models for predictive maintenance (pump degradation, sensor drift), demand forecasting per station, and water-quality prediction. To support this, the sensor sampling rate has to be raised to 1-minute resolution during active operation, and a separate raw-data collection has to be set up in the cloud database.

6.2.5 Multi-Station Cloud Federation

Expanding the current single-station design into a federated multi-station deployment with central analytics, fleet-wide reporting, and per-region performance benchmarking. The federation would also support per-station configuration, OTA firmware updates, and unified RFID user management t.

6.2.6 Automatic Plate Recognition (ALPR)

Adding a camera at the wash bay entrance, with an on-device OCR or edge-AI model, to read the Saudi vehicle plate automatically. The recognized plate is matched against the plate digits and letters stored in the users table during RFID registration, giving a second authentication factor on top of the card. This also makes it possible to bill drivers who arrive without their RFID card, and it opens the door to a fully RFID-less drive-through mode in future revisions, where the plate alone is enough to start the wash. The plate-reader inference can run on the same edge node planned for the other Edge AI workloads (Section 6.2.7), so no extra hardware is needed beyond the camera and a basic image-processing pipeline.

6.2.7 Edge AI

Moving some of the analytics anomaly detection, real-time water-quality classification onto the ESP32 itself or a more capable edge node. This cuts cloud round-trip latency and bandwidth, and it also enables a degraded-mode operation when cloud connectivity drops.

6.2.8 Combined Impact Target

Taken together, the water-circularity items (Sections 6.1.1–6.1.3) are projected to bring per-wash fresh-water consumption to under 15 L per car that’s another 60 to 70 percent cut

on top of the current implementation, putting BAREEQ at fully circular, water-positive territory. The system and analytics items (Sections 6.1.4–6.1.6) bring the operational intelligence side that's needed to manage a nationwide rollout efficiently and reliably.

6.3 Challenges Encountered

Several engineering issues came up during the design, integration, and field testing of the BAREEQ prototype. Each one was diagnosed step by step, fixed through a mix of hardware and firmware adjustments, and verified under realistic conditions. The next subsections walk through the most important issues, what they were, what caused them, what was done about them, and how the fix was performed.

6.3.1 Electromagnetic Interference (EMI) from DC Pump Motors

During initial integration, the 12 V DC diaphragm pump produced a lot of electromagnetic noise during start and stop transitions. That noise showed up as spurious pulses on the YF-S201 flow sensor's ISR, which inflated the flow readings and sometimes triggered false cycle-completion events.

Brushed-DC commutation generated high-frequency transients that coupled into the sensor signal line through both radiated and conducted electromagnetic paths.

Mitigation:

1. Implemented an 800 microsecond debounce / glitch filter in the ISR to reject sub-threshold pulses.
2. Installed 100 nF ceramic decoupling capacitors across the motor terminals to suppress brush noise at the source.
3. Added a flyback diode (1N4007) across the pump relay coil to absorb inductive kickback.

Result: Flow measurement error reduced from approximately $\pm 15\%$ to under $\pm 2\%$ under all pump-switching conditions.

6.3.2 Wi-Fi Signal Degradation Inside Metal Enclosure

The first build placed the ESP32-S3 inside a metal-heavy enclosure, and the Wi-Fi reliability dropped a lot. The signal level fell from about -55 dBm in the open to -82 dBm inside the box. That caused intermittent disconnects from the MQTT broker and dropped telemetry messages.

Root cause: Metallic shielding effect of the enclosure attenuated the 2.4 GHz signal and detuned the on-board PCB antenna.

Mitigation:

1. Repositioned the ESP32-S3 antenna near a non-metallic enclosure opening.
2. Added an external 2.4 GHz IPEX antenna with 3 dBi gain.
3. Configured automatic Wi-Fi reconnection with exponential backoff in the firmware to recover gracefully from transient drops.

Result: Average RSSI improved to approximately -62 dBm; Wi-Fi uptime increased from 91% to 99.6% during 24-hour soak testing.

6.3.3 MQTT-TLS Certificate Memory Management

Adding MQTT over TLS meant embedding the HiveMQ root CA certificate on the ESP32-S3. The first try produced out-of-memory errors during the TLS handshake on reconnect. Over time, that caused occasional broker connection failures.

Root cause: Heap fragmentation following repeated TLS session establishment, combined with an oversized default handshake buffer.

Mitigation:

1. Embedded the HiveMQ root CA in flash memory (PROGMEM) instead of dynamic heap allocation.
2. Tuned the TLS handshake buffer size from 16 KB down to 8 KB after profiling actual session payloads.
3. Released TLS context buffers immediately after successful handshake completion.

Result: Reconnection memory footprint reduced by approximately 40%; no OOM faults observed across 1,000 reconnection cycles.

6.3.4 Google Cloud Run Cold-Start Latency

The Flask backend on Google Cloud Run had cold-start delays of 2 to 4 seconds when traffic was sparse. Now and then, the dashboard's first request after an idle period would time out. That hurt the operator experience.

Root cause: Default serverless scaling policy permits container shutdown during idle periods, requiring full re-initialization on the next request.

Mitigation:

1. Configured minimum-instance scaling = 1 to keep one container warm at all times.
2. Enabled CPU-always-allocated mode for faster response under low load.
3. Added client-side request retry with a 3-second timeout fallback.

Result: First-request latency reduced from 2–4 seconds to consistently below 350 ms.

6.3.5 TDS Sensor Temperature Drift

The MAX6675-based TDS readings drifted with water temperature. The result was inconsistent water-quality classification across different times of day and varying ambient condition.

Root cause: Inherent temperature coefficient of the conductivity sensing element, which is non-negligible across the 15–45 °C operating range.

Mitigation:

1. Implemented a temperature-compensation algorithm using a thermistor reference and a linear correction coefficient of $\alpha = 0.02$ per °C.
2. Recommended factory calibration at 25 °C and on-site re-calibration prior to production deployment.

Result: TDS reading variance reduced from approximately $\pm 18\%$ to $\pm 4\%$ across the full operating temperature range.

6.3.6 ADC Noise on TDS Analog Line

The ESP32-S3 ADC produced about ± 25 mV peak-to-peak of noise on the TDS analog input. Some of that came from internal ADC nonlinearity and some from environmental EMI.

Root cause: ESP32 ADC nonlinearity is well-documented when Wi-Fi is active, particularly on shared ADC2 channels.

Mitigation:

1. Applied 12-sample moving-average filtering in firmware to smooth high-frequency noise.
2. Added a passive RC low-pass filter ($R = 1 \text{ k}\Omega$, $C = 100 \text{ nF}$, $f_c \approx 1.6 \text{ kHz}$) at the ADC input.
3. Restricted analog inputs to ADC1 channels only, which remain stable when Wi-Fi is active.

Result: ADC reading stability improved by approximately 85%; effective resolution increased from roughly 7 bits to 10 bits.

#	Challenge	Root Cause	Mitigation	Outcome
1	EMI on flow sensor	Pump commutation noise	800 μ s ISR filter + 100 nF caps + flyback diode	Error $\pm 15\% \rightarrow \pm 2\%$
2	Wi-Fi degradation	Metal enclosure shielding	External IPEX antenna + auto-reconnect	Uptime 91% $\rightarrow$ 99.6%
3	TLS out-of-memory	Heap fragmentation	Flash-embedded CA + tuned handshake buffer	Zero OOM in 1,000 cycles
4	Cloud Run cold start	Idle container shutdown	Min-instances = 1 + CPU always allocated	2–4 s $\rightarrow$ < 350 ms
5	TDS temperature drift	Sensor thermal coefficient	Compensation algorithm + 25 $^{\circ}$ C calibration	Variance $\pm 18\% \rightarrow \pm 4\%$
6	ADC analog noise	EMI + ADC nonlinearity	12-sample averaging + RC low-pass filter	Effective bits 7 $\rightarrow$ 10

Table 9 Summary of Engineering Challenges and Resolutions

6.4 Lessons Learned

The issues hit during BAREEQ's development reinforced a handful of core engineering principles that apply directly to industrial IoT deployments:

1. **Mitigate noise at the source** before resorting to firmware-level filtering hardware fixes are
2. typically, more robust and lower in CPU overhead.
3. **Design enclosures with RF transparency in mind** when wireless connectivity is critical, particularly for sub-2.4 GHz performance.
4. **Validate cloud cost-versus-latency trade-offs early**, as free-tier and serverless services often have hidden behavioral limits.
5. **Sensor calibration must account for environmental variation**, not just point accuracy at a single reference temperature.
6. **Layered mitigation (hardware combined with firmware)** consistently outperforms single-layer fixes in noisy industrial environments.

These resolved issues feed straight into the production-readiness recommendations covered in Section 6 (Conclusion and Future Work), where the path from prototype to a hardened multi-station deployment is laid out

7 References

- [1] Superoperator, "How does IoT technology benefit car wash operations?" 2025. [Online]. Available: <https://www.superoperator.com/how-does-iot-technology-benefit-car-wash-operations/>
- [2] Azilen, "Top IoT Protocols Explained (With Use Cases)," 2024. [Online]. Available: <https://www.azilen.com/blog/iot-protocols/>
- [3] Renew Car Care, "5 Common Problems Impacting Your Car Wash's Performance," 2024. [Online]. Available: <https://www.renewcarcare.com/5-common-problems-impacting-your-carwashes-performance/>
- [4] Everwash Partners, "Using predictive insights to optimize smart car wash growth," 2022. [Online]. Available: <https://www.everwashpartners.com/blog/>
- [5] KKE Wash, "5 Ways IoT is Changing the Car Wash Industry," 2020. [Online]. Available: <https://kkewash.com/en-us/internet-of-things-car-wash-industry/>
- [6] EMQ, "8 IoT Protocols and Standards Worth Exploring in 2024," 2024. [Online]. Available: <https://www.emqx.com/en/blog/iot-protocols-mqtt-coap-lwm2m>
- [7] Cloud10 Smartwash, "What Is A Smartwash And How Cloud10 Is Revolutionizing The Car Care Industry," 2024. [Online]. Available: <https://cloud10smartwash.com/>
- [8] Sensor Partners, "Sensors in automated car wash," 2025. [Online]. Available: <https://sensorpartners.com/en/applications/sensors-in-automated-car-wash/>
- [9] Lemberg Solutions, "Top 7 Smart Home Protocols Compared," 2025. [Online]. Available: <https://lebergolutions.com/blog/smart-home-protocols-explained>
- [10] TSG Solutions, "The Smart Wash Revolution Karcher vehicle wash solutions," 2025. [Online]. Available: <https://www.tsg-solutions.com/uk/news-smart-wash-revolution/>
- [11] M. Kshirsagar, K. Inamdar, A. Shinde, and M. N. Kadam, "Smart car washing center using IoT based," *Int. J. Sci. Res. Dev. (IJSRD)*, vol. 7, no. 2, pp. 2060-2064, 2019.
- [12] A. Hossain, H. Mural, A. Dash, M. F. Rahman, and M. F. Kabir, "Fabrication of automatic car washing system with multiple features using IoT," in *Proc. 7th Bangladesh Conf. Industrial Engineering and Operations Management (IEOM)*, Dec. 2024, DOI: 10.46254/BA07.20240090.
- [13] S. Zhong, L. Zhang, H. C. Chen, H. Zhao, and L. Guo, "Study of the patterns of automatic car washing in the era of Internet of Things," in *Proc. 31st IEEE Int. Conf. Advanced Information Networking and Applications Workshops (WAINA)*, 2017, pp. 82-86, DOI: 10.1109/WAINA.2017.132.

- [14] H. Al Banna, S. Ashif, M. Sanaullah, H. Patwary, S. Hossain, and A. M. Ibrahim, "Design of a password protected automatic car washing system," in Proc. 2020 Int. Conf. Computer, Electrical and Communication Engineering (ICCECE), 2020.
- [15] P. Brous, M. Janssen, and P. Herder, "The dual effects of the Internet of Things (IoT): A systematic review of the benefits and risks of IoT adoption by organizations," *Int. J. Information Management*, vol. 51, p. 101952, 2020, DOI: 10.1016/j.ijinfomgt.2019.05.008.
- [16] H. Janik and A. Kupiec, "Trends in modern car washing," *Polish J. of Environmental Studies*, vol. 16, no. 6, pp. 927-931, 2007.
- [17] U.S. Environmental Protection Agency (EPA), *WaterSense at Work: Best Management Practices for Commercial and Institutional Facilities Section 5.5: Vehicle Washing*, Washington, DC, Oct. 2012.
- [18] F. Jan, N. Min-Allah, S. Saeed, S. Z. Iqbal, and R. Ahmed, "IoT-based solutions to monitor water level, leakage, and motor control for smart water tanks," *Water*, vol. 14, no. 3, p. 309, 2022.
- [19] B. VeerasekharReddy et al., "Water quality monitoring system using IoT and cloud," in Proc. 2023 Int. Conf. Sustainable Computing and Smart Systems (ICSCSS), pp. 1036-1042, IEEE, 2023.
- [20] D. Ramegowda and M. Lin, "Energy efficient mixed task handling on real-time embedded systems using FreeRTOS," *J. Systems Architecture*, vol. 131, p. 102708, 2022.
- [21] Badusha. S, Gopikannan. S, Janarthanan. S, and Gnanasekaran. S, "Waste Water Management for Smart City," *Int. J. Trend Sci. Res. Dev. (IJTSRD)*, vol. 4, no. 2, pp. 1110-1113, Feb. 2020. [Online]. Available: <https://www.ijtsrd.com/papers/ijtsrd30231.pdf>
- [22] M. Al-Sayed et al., "IoT-Based Smart Water Management Systems for Residential Buildings in Saudi Arabia," *Processes*, vol. 10, no. 11, p. 2462, Nov. 2022, DOI: 10.3390/pr10112462.
- [23] R. Alotaibi et al., "Internet of Things (IoT)-Based Wastewater Management in Smart Cities," *Electronics*, vol. 12, no. 12, p. 2590, Jun. 2023, DOI: 10.3390/electronics12122590.
- [24] S. Ismail, D. W. Dawoud, N. Ismail, R. Marsh, and A. S. Alshami, "IoT-based water management systems: survey and future research direction," *IEEE Access*, vol. 10, pp. 35942-35952, 2022, DOI: 10.1109/ACCESS.2022.3163918.

- [25] G. Marques, R. Pitarma, N. M. Garcia, and N. Pombo, "Open-Source MQTT-Based End-to-End IoT System for Smart City Scenarios," *Future Internet*, vol. 14, no. 2, p. 57, Feb. 2022, DOI: 10.3390/fi14020057.
- [26] J. Ali, M. H. Zafar, C. Hewage, R. Hassan, and R. Asif, "Mathematical Modeling and Validation of Retransmission-Based Mutant MQTT for Improving Quality of Service in Developing Smart Cities," *Sensors*, vol. 22, no. 24, p. 9751, Dec. 2022, DOI: 10.3390/s22249751.

8 Appendices

8.1 Source Code

The complete source code for the Bareeq system is publicly available on GitHub at This [LINK](#)

8.2 Appendix A: ESP32-S3 Pin Assignment

Table 9 lists the complete ESP32-S3 GPIO pin assignment used in the BAREEQ prototype

GPIO Pin	Function	Component	Protocol
GPIO 10	RFID SS (Chip Select)	MFRC522	SPI
GPIO 11	RFID MOSI	MFRC522	SPI
GPIO 12	RFID SCK	MFRC522	SPI
GPIO 13	RFID MISO	MFRC522	SPI
GPIO 14	RFID RST	MFRC522	GPIO
GPIO 8	I2C SDA	AHT20	I2C
GPIO 9	I2C SCL	AHT20	I2C
GPIO 4	Air flow ISR	YF-S201 (air)	Interrupt
GPIO 5	Water flow ISR	YF-S201 (water)	Interrupt
ADC 1	TDS analog input	TDS Sensor	ADC 12-bit
ADC 3	Turbidity analog input	Turbidity Sensor	ADC 12-bit
GPIO 2	MAX6675 CS	Thermocouple amp	HSPI
GPIO 36	MAX6675 SCK	Thermocouple amp	HSPI
GPIO 37	MAX6675 MISO	Thermocouple amp	HSPI
GPIO 17	Water pump relay	Relay Module 1	Digital OUT
GPIO 45	Air pump relay	Relay Module 2	Digital OUT

Table 10: ESP32-S3 Pin Assignment for BAREEQ Prototype

8.3 Appendix B: MQTT Topic Structure

The MQTT topic hierarchy and JSON payload structure used between ESP32 firmware and the cloud backend are detailed in Table 10.

Topic	Direction	Payload (JSON fields)
carwash/station-01/wallet/request	ESP32 to Broker	req_id, uid
carwash/station-01/wallet/response	Broker to ESP32	req_id, uid, service, approved, price, new_balance, reason
carwash/station-01/mission/summary	ESP32 to Broker	ts, station, device, uid, req_id, mission: {total_air_l, total_water_l, avg_env_temp_c, avg_humidity_pct, avg_water_temp_c, avg_tds_ppm, avg_turbidity_ntu, samples}

Table 11: BAREEQ MQTT Topic and Payload Structure

8.4 Appendix C: Service Pricing and Loyalty

Table 11 lists the three available service codes with their corresponding base prices in Saudi Riyal (SAR).

Service Code	Base Price (SAR)	Description
OUT_WASH	20	Exterior wash only default service
IN_OUT_WASH	30	Interior and exterior wash
IN_OUT_WASH_POLISH	100	Full interior/exterior wash with polish

Table 12: BAREEQ Service Codes and Pricing

8.5 Appendix D: Firmware Key Code Sections

```
// FreeRTOS task creation all pinned to Core 1, priority 2

xTaskCreatePinnedToCore(taskRelays, "relays", 4096, NULL, 2,
&taskRelayHandle, 1);

xTaskCreatePinnedToCore(taskSensors, "sensors", 6144, NULL, 2,
&taskSensorsHandle, 1);

xTaskCreatePinnedToCore(taskRFID, "rfid", 6144, NULL, 2,
&taskRFIDHandle, 1);

// Wash sequence timing constants

static const uint32_t WATER_MS = 30000; // 30 seconds
```

```

static const uint32_t BREAK_MS = 5000; // 5 seconds
static const uint32_t AIR_MS = 30000; // 30 seconds
// TDS calculation (12-sample averaged ADC)
float voltage = adcReading * (3.3f / 4095.0f);
float tds_ppm = voltage * 500.0f;

```

8.6 Appendix E: Backend REST API Reference

Method	Endpoint	Description
GET	/api/health	System health check
GET	/api/stations/services	Get service type and pricing for all stations
POST	/api/stations/{id}/service	Set active service type for a station
GET	/api/users/{uid}	Get user details by UID
GET	/api/users/search?q=&by=	Search users by name, UID, or car plate
POST	/api/users	Register a new RFID user
POST	/api/users/add-balance	Add credit to user account
POST	/api/users/block	Block or unblock a user account
POST	/api/users/delete	Delete a user account
GET	/api/history/{station}/{metric}	Get sensor time series (range: 1h/12h/1d/1w/1m/1y)
GET	/api/analytics/overview	Annual totals: revenue, visits, water, sensors
GET	/api/analytics/monthly	Month-by-month breakdown for selected year
GET	/api/analytics/summary	Full summary including ARPU and service mix
WS	/ws	WebSocket: live mission state push every 1 second

Table 13: BAREEQ Backend REST API and WebSocket Endpoint Reference